

当代数据管理系统实验课总结（考试携带）

毕竟是开卷考试。但是也不能把考试要带过去的东西搞得过度复杂，否则肯定找不到。

MongoDB

1.连接mongoDB数据库

```
#导入库
import pymongo
#本地连接
client= pymongo.MongoClient("mongodb://localhost:27017/")
#连接mongodb
client = pymongo.MongoClient("mongodb://stu10205501417:stu10205501417@stu10205501417.mongodb.rds.aliyuncs.com:3717,dase-cdms-2022-secondary-pub.mongodb.rds.aliyuncs.com:3717/admin?replicaSet=mgset-62456703")
#切换到自己的数据库
db = client['stu10205501417']
# print 数据库名字，应是“stu学号”的格式
print(client.list_database_names())
```

[‘stu10205501417’]

```
#获取集合，如果没有则自动创建，这里设定集合是users，在python中用users_col指代
users_col = db['users']
# 等价于 users_col = db.users
users_col
```

```
#先将该集合中所有数据删除（删除操作讲解在最末尾），保证数据和lab2中第2部分插入数据后的一模一样。如有插入错误，可使用该语句重新插入
users_col.delete_many({})
```

2.插入数据

- insert_one (参数): 插入一条数据，插入的参数就是Python的字典
- insert_many(参数): 批量插入数据，参数为多个字典的列表

2.1insert_one

插入一条用户数据

```
user1={
    "name" : "xiaobu",
    "country" : "China",
    "favorites": {
        "books":["西游记", "红楼梦"],
        "cities":["韶关", "深圳" ]
    },
    "age" : 26
}
result = users_col.insert_one(user1)
```

```
#为"name"字段创建唯一索引约束，保证同一name的文档只有一条能够插入。如果重复插入同一name的文档，则会报错，无法插入。
users_col.create_index([("name", 1)], unique=True)
```

如果直接对插入的操作打印(print(result)), 返回的结果是pymongo.results.InsertOneResult格式.需要使用下面的语句才能成功输出数据表中的信息:

```
content = users_col.find()
for each in content:
    print(each)
```

```
{'_id': ObjectId('632964737c60abfd46f5408c'), 'name': 'xiaobu', 'country': 'China',.....}
```

显示一条数据，即插入成功。

2.insert_many

插入多条数据：

```
user2={
    "name" : "tommy",
    "country" : "China",
    "favorites": {
        "books":["西游记", "红楼梦"],
        "cities":["韶关", "深圳" ]
    },
    "age" : 26
}
user3={
    "name" : "jack",
    "country" : "China",
    "favorites": {
        "books":["西游记", "红楼梦"],
        "cities":["韶关", "深圳" ]
    },
    "age" : 26
}
users=[user2,user3]
users_col.insert_many(users)
content = users_col.find()
for each in content:
    print(each)
```

3.查找数据

MongoDB的查找功能对应的方法是

- find(查询条件, 返回字段)
- find_one(查询条件, 返回字段): 一次只返回一次信息

返回字段为一个字典 {key:value}, key为字段名称, value为 0 (不返回该字段) 或 1 (返回该字段)。其中_id是一个特殊字段, 只有指定_id:0 才不会返回, 否则默认返回。

3.1基础查询

find()返回的是可迭代的PyMongo对象, 这个对象可以被for循环展开。展开后得到每一条都是记录的字典。

查找年龄为25岁, 并且只显示名字。

```
result = users_col.find({'age': 25}, {'_id': 0, 'name': 1})
for each in result:
    print(each)
```

{'name': 'juyi'}

还可以以下面这种形式输出：

```
content = [x for x in users_col.find({'age': 25}, {'_id': 0, 'name': 1})]
print(content)
```

[{'name': 'juyi'}]

显示所有数据（在前面多次使用）：

```
for each in users_col.find():
    print(each)
```

查询country为China的用户(要求：输出为列表)：

```
content = [x for x in users_col.find({'country': 'China'}, {'_id': 0, 'name': 1})]
print(content)
```

```
[{'name': 'xiaobu'}, {'name': 'juyi'}, {'name': 'lilei'}, {'name': 'zhangsan'}]
```

查询年龄在20岁并且身高为1.71m的用户

```
result = users_col.find({'age':20,'height':1.71},{'_id':0})
```

查询所有age > 25的记录，并且不显示_id

```
results = users_col.find({"age": {"$gt": 25}}, {"_id": 0})
```

查询用户的country是USA并且年龄大于20岁的数据。

```
results = users_col.find({"country": "USA", "age": {"$gt": 20}})
```

查找喜欢书籍的，且身高在1.68-1.72m之间的中国用户

```
query = {
  "country": "China",
  "favorites.books": {"$exists": True},
  "height": {"$gte": 1.68, "$lte": 1.72}
}
results = users_col.find(query)
```

3.2逻辑查询

符号	含义	示例
\$lt	小于	{'age': {'\$lt': 20}}
\$gt	大于	{'age': {'\$gt': 20}}
\$lte	小于等于	{'age': {'\$lte': 20}}
\$gte	大于等于	{'age': {'\$gte': 20}}
\$ne	不等于	{'age': {'\$ne': 20}}
\$in	在范围内	{'age': {'\$in': [20, 23]}}
\$nin	不在范围内	{'age': {'\$nin': [20, 23]}}
\$or	逻辑非	{'\$or': [{'age':25},{'country':'USA'}]}
\$and	逻辑与	{'\$and': [{'age':25},{'country':'USA'}]}

查询用户年龄在21和25之间的记录（包括21和25岁）,并且不显示_id：

```
result = users_col.find({'age': {'$gte':21, '$lte':25}},{'_id':0})
```

查询favorites.movies中，只有“头号玩家”,“肖生克的救赎”的用户的名字，并且喜欢的电影以该顺序排序:

```
result = users_col.find({
  'favorites.movies': ['头号玩家','肖生克的救赎']
},{ '_id':0,'name':1}
)
```

查找喜欢书籍“谁动了我的奶酪” 或者 年龄不小于25岁且家不在纽约的美国用户

```
query = {
  "$or": [
    {"favorites.books": "谁动了我的奶酪"},
    {"$and": [{"age": {"$gte": 25}}, {"address.aName": {"$ne": "纽约"}}, {"country": "USA"} ]}
  ]
}
results = users_col.find(query)
```

查找既喜欢cites也喜欢books，且身高在1.80以上的用户

```

query = {
  "$and": [
    {"favorites.cites": {"$exists": True}},
    {"favorites.books": {"$exists": True}},
    {"height": {"$gt": 1.80}}
  ]
}
results = users_col.find(query)

```

3.3匹配查询

符号	含义	示例	示例含义
\$regex	匹配正则表达式	{'name': {'\$regex': '^M.*'}}	name以M开头
\$exists	属性是否存在	{'name': {'\$exists': True}}	name属性存在
\$type	类型判断	{'age': {'\$type': 'int'}}	age的类型为int
\$mod	数字模操作	{'age': {'\$mod': [5, 0]}}	年龄模5余0
\$text	文本查询	{'text': {'\$search': 'Mike'}}	text类型的属性中包含Mike字符串
\$where	高级条件查询	{'\$where': 'obj.fans_count == obj.follows_count'}	自身粉丝数等于关注数

查询用户名字以'xiao'开头的的数据

```
results = users_col.find({'name': {'$regex': '^xiao.*'}}, {'_id': 0})
```

查找名字以xiao开头，且年龄在20-25岁之间的用户

```

query = {
  "name": {"$regex": "^xiao"},
  "age": {"$gte": 20, "$lte": 25}
}
results = users_col.find(query)

```

3.4排序

sort(): 在其中传入排序的字段及升降序标志 `collection.find().sort('列名', 升降序标志)` `pymongo.ASCENDING` 指定升序(或者1) ; `pymongo.DESCENDING` 指定降序 (或者-1) ;

```

results = users_col.find().sort('name', pymongo.ASCENDING)
print([result['name'] for result in results])

```

```
results = users_col.find().sort('name', -1)
```

3.5偏移

- skip(x): 偏移x个位置，忽略前x个元素，得到第x+1个及以后的元素

```
results = users_col.find().sort('name', pymongo.ASCENDING).skip(2)
```

- limit(): 指定要取的结果个数

```
results = users_col.find().sort('name', pymongo.ASCENDING).skip(2).limit(2)
```

在数据库数量非常庞大的时候，如千万、亿级别，最好不要使用大的偏移量来查询数据，因为这样很可能导致内存溢出。

查找用户中身高按降序排序中，排名4-6的用户

```
results=users_col.find().sort('height',-1).skip(3).limit(3)
```

4.更新

- collection.update_one(参数1, 参数2): 只更新一条信息
- collection.update_many(参数1, 参数2): 更新所有符合要求的信息

参数1和参数2都是字典, 都不能省略。参数1用来寻找需要更新的记录, 参数2用来更新记录的内容。

将名字为xiaoxiao的人的身高改为1.81(使用\$set)

```
condition = {'name': 'xiaoxiao'}
users_col.update_one(condition, {'$set':{'height': 1.81}})
```

将年龄大于20岁的用户的年龄增加1岁 (使用\$inc)

```
condition = {'age': {'$gt': 20}}
result = users_col.update_many(condition, {'$inc': {'age': 1}})
```

将喜欢书籍且在中国的用户, 所在城市改为上海

```
query = {
    "country": "China",
    "favorites.books": {'$exists': True}
}
update = {
    "$set": {
        "address.aName": "上海"
    }
}
result = users_col.update_many(query, update)
```

对查询结果去重

- collection.distinct('列名'): 返回一个去重后的列表

```
name = users_col.distinct('name')
```

用户的年龄有多少种(最后输出数字,提示: 可使用python的len())

```
age = users_col.distinct('age')
print(len(age))
```

5.删除

- collection.delete_one(参数): 删除一条记录
- collection.delete_many(参数): 删除符合要求的所有记录

参数都是字典, 不建议省略。

删除第1个country在USA的用户

```
users_col.delete_one({'country': 'USA'})
result = users_col.find({}, {'_id': 0})
```

删除所有country在USA或者年龄小于等于26岁的用户

```
query = {
    "$or": [
        {"country": "USA"},
        {"age": {"$lte": 26}}
    ]
}
users_col.delete_many(query)
```

删除所有数据

```
users_col.delete_many({})
```

6.Python Object与Mongodb Document的转换

ODM(Object-Document Mapper)是指将python中的对象和MongoDb中文档之间进行映射，将文档和对文档数据的操作封装成一个python类。用户在操作文档数据的时候，只需关心类的定义与使用方法即可，无需关系与mongodb的交互细节。

如下是一个ODM操作示例：

创建一个userTable类，python_to_mongo函数将类对象转化为文档插入mongodb，mongo_to_python函数根据查询条件从mongodb中查询出相应的文档并将其转化为python类对象

```
class userTable(object):
    def __init__(self, _id, name, age, country, height, favorites):
        self._id = _id
        self.name = name
        self.age = age
        self.country = country
        self.height = height
        self.favorites = favorites

# 插入一条文档
def insert_one_to_mongo(user):
    users_col.insert_one(user.__dict__)

# 根据条件从mongodb中查询文档
def query_from_mongo(condition):
    result = users_col.find(condition)
    user_list = []
    for doc in result:
        temp = userTable(doc['_id'], doc['name'], doc['age'], doc['country'], doc['height'], doc['favorites'])
        user_list.append(temp)
    return user_list

if __name__ == '__main__':

    #首先删除users_col全部的数据
    users_col.delete_many({})

    #构造两条数据
    user1 = userTable('001', 'lilei', 28, 'China', 1.81, {'books':['谁动了我的奶酪', '飘']})
    user2 = userTable('002', 'hanmeimei', 30, 'China', 1.70, {'movies':['头号玩家', '肖申克的救赎']})
    user3 = userTable('003', 'zhangsan', 21, 'China', 1.76, {'books':['人类简史', 'CASPP']})

    #插入三条数据
    insert_one_to_mongo(user1)
    insert_one_to_mongo(user2)
    insert_one_to_mongo(user3)

    #根据查询条件查询文档（查询喜欢书籍的用户）
    user_list = query_from_mongo({'favorites.books':{'$exists':True}})

    #输出结果
    for s in user_list:
        print(s.__dict__) #lilei & zhangsan
```

```
{'_id': '001', 'name': 'lilei', 'age': 28, 'country': 'China', 'height': 1.81, 'favorites': {'books': ['谁动了我的奶酪', '飘']}}
{'_id': '003', 'name': 'zhangsan', 'age': 21, 'country': 'China', 'height': 1.76, 'favorites': {'books': ['人类简史', 'CASPP']}}
```

根据示例程序，完成下述程序待填充的部分。 两个函数：update_to_mongo、delete_many_to_mongo 四个CRUD操作：

```
class userTable(object):
    def __init__(self, _id, name, age, country, height, favorites):
        self._id = _id
        self.name = name
        self.age = age
        self.country = country
        self.height = height
        self.favorites = favorites

# 插入一条文档
def insert_one_to_mongo(user):
```

```

users_col.insert_one(user.__dict__)

# 根据条件从mongodb中查询文档
def query_from_mongo(condition):
    #查询
    result = users_col.find(condition)
    #将查询结果转换为python对象
    user_list = []
    for doc in result:
        temp = userTable(doc['_id'],doc['name'],doc['age'],doc['country'],doc['height'],doc['favorites'])
        user_list.append(temp)
    return user_list

#根据条件和更新操作，对文档进行更新
def update_to_mongo(condition, operation):
    #更新
    users_col.update_many(condition, operation)

#根据条件删除文档
def delete_many_to_mongo(condition):
    #删除
    users_col.delete_many(condition)
    result = users_col.find()
    user_list = []
    for doc in result:
        temp = userTable(doc['_id'],doc['name'],doc['age'],doc['country'],doc['height'],doc['favorites'])
        user_list.append(temp)
    return user_list

if __name__=='__main__':

    #首先删除users_col全部的数据
    delete_many_to_mongo({})

    #构造两条数据
    user1 = userTable('001', 'lilei', 28, 'China', 1.81, {'books':['谁动了我的奶酪','飘']})
    user2 = userTable('002', 'hanmeimei', 30, 'China',1.70, {'movies':['头号玩家','肖申克的救赎']})
    user3 = userTable('003', 'zhangsan', 21, 'China',1.76, {'books':['人类简史','CASPP']})

    #插入两条数据
    insert_one_to_mongo(user1)
    insert_one_to_mongo(user2)
    insert_one_to_mongo(user3)

    #根据查询条件查询文档（查询年龄大于25岁且身高高于1.75的用户）
    query = {
        'age': {'$gt':25},
        'height': {'$gt':1.75}
    }
    user_list = query_from_mongo(query)    #待填充

    #输出结果
    print('修改之前\n')
    for s in user_list:
        print(s.__dict__) #lilei

    query = {
        'name': 'hanmeimei',
    }
    ope = {
        "$set": {
            "height": 1.76
        }
    }
    #将hanmeimei的身高改为1.76。
    update_to_mongo(query,ope)                #待填充

    #根据查询条件查询文档（查询年龄大于25岁且身高高于1.75的用户）
    query = {
        'age': {'$gt':25},
        'height': {'$gt':1.75}

```

```
}
user_list = query_from_mongo(query)    #待填充

print('\n修改之后\n')
#输出结果
for s in user_list:
    print(s.__dict__) #lilei & hanmeimei

#删除喜欢书籍的用户
query = {
    'favorites.books':{'$exists': True}
}
delete_many_to_mongo(query)           #待填充

#查询剩余全部用户
user_list = query_from_mongo({})

print('\n删除之后\n')
#输出结果
for s in user_list:
    print(s.__dict__) #hanmeimei
```

7.聚合

何为聚合操作？聚合操作主要是通过对数据进行分组后做出一些简单的运算，例如平均，求和，最值等。MongoDB中的聚合运算主要通过aggregate()方法实现python中aggregate()方式实现了利用聚合管道对文档进行变化计算和展示。文档进入聚合管道会依次经过筛选(filtering)，分组（grouping）并聚合，排序（sorting），投射(projecting)，限制（limiting）（或者跳过（skipping））变化。

符号	含义	描述
\$match	筛选	按照一定条件筛选出特定的文档记录
\$project	选择	修改输入文档的结构。可以用来重命名、增加或删除域，也可以用于创建计算结果以及嵌套文档
\$group	分组	对文档按照字段分组，以便做一些聚合运算
\$sort	排序	对文档按照字段排序
\$limit	限制	限制MongoDB聚合管道返回的文档数
\$skip	跳过	在聚合管道中跳过指定数量的文档，并返回余下的文档

按照城市分组计数

```
result = users_col.aggregate([
    {'$group':{'_id':'$city','count':{'$sum':1}}}
])

{'_id': 'Shanghai', 'count': 3}
{'_id': 'Beijing', 'count': 1}
{'_id': 'Nanjing', 'count': 1}
{'_id': 'Chengdu', 'count': 3}
{'_id': 'Hangzhou', 'count': 1}
```

从以上代码中，聚合管道中只有group一个操作。在group中，可以看到是按照city字段进行分组，最后通过“加一”聚合来实现分组计数的。在group中指定了两个字段，第一个是主键_id，来源于city，第二个是count，来源于求和。当然也可以根据我们的需要，修改/增减字段。

以下列出了常用的聚集运算

符号	含义
\$sum	求和
\$avg	求平均
\$min	最小值
\$max	最大值
\$push	聚合成数组

符号	含义
\$addToSet	聚合成几何
\$first	排序取第一个
\$last	排序取最后一个

计算不同性别用户的平均年龄，最大年龄，最小年龄并且输出

```
pipeline = [
  {
    "$group": {
      "_id": "$gender",
      "avg_age": {"$avg": "$age"},
      "max_age": {"$max": "$age"},
      "min_age": {"$min": "$age"}
    }
  }
]
result = users_col.aggregate(pipeline)
```

列出不同性别的学生名单 (\$push)

```
pipeline = [
  {
    "$group": {
      "_id": "$gender",
      "students": {"$push": "$name"}
    }
  }
]
result = users_col.aggregate(pipeline)
```

复杂查询

一个完整的查询一般需要经过：

- 1、通过条件筛选文档记录（选择文档的行记录）
- 2、分组并聚合
- 3、对文档按照某些字段排序
- 4、调整文档的键值对形式（调整文档的列）
- 5、通过limit或者skip展示特定数量的记录

筛选年龄大于等于20岁的同学，并将这些学生的按照城市分组计算平均年龄后升序排列,显示城市和平均年龄

```
match={'$match':{'age':{'$gte':20}}}
group={'$group':{'_id':'$city','avg_age':{'$avg':'$age'}}}
sort={'$sort':{'avg_age':1}}#1代表升序，-1代表降序
project={'$project':{'avg_age':1}}
result=users_col.aggregate([match,group,sort,project])
for each in result:
    print(each)
```

```
{'_id': 'Nanjing', 'avg_age': 22.0}
{'_id': 'Hangzhou', 'avg_age': 22.0}
```

格式化输出每个同学的生日日期，按照生日日期排序

```
sort={'$sort':{'birthdate.year':-1,'birthdate.month':-1,'birthdate.day':-1}}
project={'$project':{'_id':0,'name':1,'birthday':{'$concat':[
    {'$toString': "$birthdate.year"},'-',
    {'$toString': "$birthdate.month"},'-',
    {'$toString': "$birthdate.day"}]}}}#0代表不显示该字段，1代表显示该字段
result = users_col.aggregate([sort,project])
```

```
{'name': 'Jason', 'birthday': '1999-12-12'}
{'name': 'Donna', 'birthday': '1998-9-9'}
{'name': 'Kate', 'birthday': '1998-7-7'}
```

【以上的toString的作用是将数字转换成字符串，更多函数可以参见https://blog.csdn.net/weixin_43632687/article/details/104201185】

按照城市，性别分组计数

```
group={'$group':{'_id':{'city':'$city','gender':'$gender'},'count':{'$sum':1}}}
project={'$project':{'_id':0,'city':'$_id.city','gender':'$_id.gender','count':'$count'}}
result = users_col.aggregate([group,project])
```

找出喜欢'violin'的人数（提示：\$in）

```
match={'$match':{'hobby':{'$in':['violin']}}}
project={'$project':{'_id':'like_violin','count':'$count'}}
group={'$group':{'_id':'like_violin','count':{'$sum':1}}}
result = users_col.aggregate([match,group,project])
```

\$unwind拆分数组，查询拥有各个爱好的学生人数

```
#先通过数组拆分将一条记录拆分成多条记录
unwind={'$unwind':'$hobby'}
group={'$group':{'_id':'$hobby','count':{'$sum':1}}}
result = users_col.aggregate([unwind,group])
```

找出爱好个数为3的同学，展示姓名，年龄与爱好（不使用\$size来求长度，要求使用unwind来拆分数组和push来合并数组）

```
pipeline = [
    {'$unwind': '$hobby'},
    {
        '$group': {
            '_id': {
                'name': '$name',
                'age': '$age'
            },
            'count': {'$sum': 1},
            'hobbies': {'$push': '$hobby'}
        }
    },
    {'$match': {'count': {'$in': [3,3]} }},
    {
        '$project': {
            '_id': 0,
            'name': '$_id.name',
            'age': '$_id.age',
            'hobby': '$hobbies'
        }
    }
]
result = users_col.aggregate(pipeline)
for each in result:
    print(each)
```

8.索引

单条索引：按照姓名生序

```
users_col.create_index([("name", 1)],unique=True)
```

创建复合索引，按照姓名生序，按照年龄降序

```
users_col.create_index([("name", 1),("age", -1)], unique=True)
```

'name_1_age_-1'

删除索引

```
print('删除前索引信息\n',users_col.index_information())
users_col.drop_index('name_1')#括号里面的参数是索引名
users_col.drop_index('name_1_age_-1')
print('删除后索引信息\n',users_col.index_information())
```

性能测试

索引的价值在于提高基于索引字进行段数据查询的效率，我们可以通过构造一批数据，对比建索引前的查询时间来体会索引的价值

mxc:从结果来看，创建索引之后的插入数据效率较之没有插入索引时要低，从我的测试结果来看，创建一个索引会让效率变为原来的 $\frac{2}{3}$ 。因为索引创建之后相当于在数据库内部维护了一个便于检索的结构，会增大存储空间消耗，也会加大存储结构的复杂度，但是数据检索的效率会大大提高（就像前面的实验结果一样）。因此，索引相当于数据库的一个“空间换时间”的技术，使用时需要权衡其利弊，选择合适的属性作为索引。

我：可发现创建索引后插入时间不减反增。为什么会出现这样的情况呢？

插入数据最效率高的方法就是无索引直接插入。建立索引之后，每次插入都要更新索引，把它插入到索引中的适当位置。这个额外的操作会有额外的开销，使得插入操作变慢。设计数据库时，我们可以通过建立索引来提高查询性能，但不能滥用这一特性，还是要根据实际情况去考虑

scx:结论:增加索引后，由于会使数据库添加表格对索引进行维护，因此插入时所用的时间必定会增加。原本只需要插入Tuple，如果增加索引，就需要对索引表进行额外的插入，以此延长时间。同时我们也可以看到，复合索引的复合数量越多(如name, name_age, name_age_gender进行对照)，那么所用的时间也会增加，这是由于此时索引表的Size变得更大，所以插入的代价变高了。从（name索引与name和age分别索引中可以看到）:增加索引个数会延长时间，这是由于增加索引个数会增加索引表的数量，如name索引只需要一张name索引表，而name和age分别索引需要name和age两张索引表，因此每次插入Tuple时需要额外插入两张索引表，使插入所需的时间变高

PostgreSQL

1.连接

使用psycopg2连接并操作postgresql。连接数据库，host和port不用修改，user="stu学号", password="Stu学号", database="stu学号" 获取SQL执行器cursor，后续会使用cursor执行SQL并获取返回结果。

```
# 导入python与postgresql操作库
import psycopg2
conn = psycopg2.connect(host="dase-cdms-2022.pg.rds.aliyuncs.com", port="5432", user="stu10205501417",
password="Stu10205501417", database="stu10205501417")
autocommit = psycopg2.extensions.ISOLATION_LEVEL_AUTOCOMMIT
conn.set_isolation_level(autocommit)
cur = conn.cursor()
# 列出当前所在数据库的名字
sql = "select current_database();"
# 执行SQL
cur.execute(sql)
# SQL执行后，会将结果以元组的形式缓存在cursor中，使用下述语句输出
for tuple in cur.fetchall():
    print(tuple)
```

('stu10205501417,')

创建employee表

```
sql = 'CREATE TABLE EMPLOYEE(ID INT PRIMARY KEY, NAME TEXT, AGE INT, CITY CHAR(15), SALARY REAL);'
cur.execute(sql)
```

列出当前数据库中的表，与在SQL Shell中操作完全不同（SQL Shell将一些常见指令封装为快捷操作符"\c \d \l"等，但是使用psycopg2需要写出完整的SQL命令）：

```
cur.execute("""SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public'""")

for tuple in cur.fetchall():
    print(tuple)
```

('employee,')

删除employee表的主键约束

```
sql = "ALTER TABLE EMPLOYEE DROP CONSTRAINT employee_pkey;"
cur.execute(sql)
```

查看约束的变化。目前没有找到比较方便使用的SQL语句，大家可以在终端中连接PG并查询

在employee表的id属性创建主键约束

```
sql = "ALTER TABLE EMPLOYEE ADD CONSTRAINT employee_pkey PRIMARY KEY (id);"
cur.execute(sql)
```

在终端中输入 "\d employee" 查询约束信息，可以看到主键约束已经存在。

插入一条记录，%s为占位符，在执行插入语句时填入相应的值

```
sql = "insert into employee (id, name, age, city, salary) values (%s, %s, %s, %s, %s)"

cur.execute(sql, (1, '张山', 25, '上海', 10000))
```

输出employee表中全部的元组

```
sql = "select * from employee"
cur.execute(sql)

for tuple in cur.fetchall():
    print(tuple)
```

批量插入多条数据 构造一个二维列表

```
values = [[2, '李思', 23, '北京', 12000],
          [3, '王武', 28, '深圳', 16000],
          [4, '赵柳', 22, '广州', 8000]]

sql = "insert into employee (id, name, age, city, salary) values (%s, %s, %s, %s, %s)"

cur.executemany(sql, values)
```

输出employee表中全部的元组

```
sql = "select * from employee"
cur.execute(sql)

for tuple in cur.fetchall():
    print(tuple)
```

(1, '张山', 25, '上海', 10000.0)

(2, '李思', 23, '北京', 12000.0)

查询收入在10000以上的员工姓名与年龄

```
sql = "select name, age from employee where salary > %s"
# 注意10000后面要加一个","
cur.execute(sql, (10000,))
for tuple in cur.fetchall():
    print(tuple)
```

删除employee全部的数据

```
sql1 = "Delete from employee"
sql2 = "DROP TABLE employee"
cur.execute(sql1)
cur.execute(sql2)
```

2.DDL DQL

考虑一个零件销售供应商数据库，里面包含四张数据表：S（供应商）、P（零件）、J（工厂）、SPJ（工厂订购供应商零件的信息），每张数据表的初始化属性与约束如下：

S: SNO (CHAR(2)), SNAME (CHAR(6)), STATUS (CHAR(2)), CITY (CHAR(4)) SNO为主键

P: PNO (CHAR(2)), PNAME (CHAR(6)), COLOR (CHAR(2)), WEIGHT (INT) PNO为主键

J: JNO (CHAR(2)), JNAME (CHAR(8)), CITY (CHAR(4)) JNO为主键

SPJ: SNO char(2), PNO char(2), JNO char(2) (SNO,PNO,JNO)为主键

2.1创建表与约束

在创建表前，最基础的事情是确定属性及属性的类型，除了前面的讲解中出现的与下方列出的常用数据类型外，PostgreSQL提供了非常丰富的数据类型：

类型	描述
TEXT	字符类型，变长，无长度限制
CHAR(NUM)	字符类型，定长，字符长度为NUM
VARCHAR(NUM)	字符类型，变长，字符最长限制为NUM
BOOL	布尔类型
REAL	浮点类型（单精度）
float8	浮点类型（双精度）
INT	整型

除了前面的讲解中出现的主键约束、非空约束与唯一约束外，数据库还包含外键约束和检查约束，简介如下：

约束	描述
主键约束	NOT NULL 和 UNIQUE 的结合。确保某列（或两个列多个列的结合）有唯一标识，有助于更容易更快速地找到表中的一个特定的记录。
唯一约束	在一列中或者一组列中保存的数据在表中所有行间是唯一的
非空约束	指定一个列中不会有空值
外键约束	保证一个表中的数据匹配另一个表中的值的参照完整性
检查约束	指定一个某个列的值必须要满足一个布尔表达式

根据上述表的属性和约束信息，完成表的创建：

```
# 建表sql
create_table_S = 'CREATE TABLE S(SNO CHAR(2) PRIMARY KEY, SNAME CHAR(6), STATUS CHAR(2), CITY CHAR(4));'
create_table_P = 'CREATE TABLE P(PNO CHAR(2) PRIMARY KEY, PNAME CHAR(6), COLOR CHAR(2), WEIGHT INT);'
create_table_J = 'CREATE TABLE J(JNO CHAR(2) PRIMARY KEY, JNAME CHAR(8), CITY CHAR(4));'
create_table_SPJ = 'CREATE TABLE SPJ(SNO char(2), PNO char(2), JNO char(2), PRIMARY KEY (SNO, PNO, JNO));'
# 执行sql
cur.execute(create_table_S)
cur.execute(create_table_P)
cur.execute(create_table_J)
cur.execute(create_table_SPJ)

cur.execute("""SELECT table_name FROM information_schema.tables
              WHERE table_schema = 'public'""")
for tuple in cur.fetchall():
    print(tuple)
```

('s,)
('p',)
('j',)
('spj',)

2.2DDL修改表信息

如果我们在创建表后，发现表信息需要修改或者需要修改约束条件，那么就需要利用DDL语句修改表。

常见的修改表语句如下所示：

操作	语法格式
增加\删除列	ALTER TABLE tablename ADD COLUMN colname coltype ALTER TABLE tablename DROP COLUMN colname

操作	语法格式
增加\删除约束	xxxxx ADD CONSTRAINT constraint_name constraint_type (col_name)(主键\UNIQUE约束) xxxxx ADD FOREIGN KEY (colname) REFERENCES tablename(外键约束) xxxxx ALTER COLUMN colname SET NOT NULL(非空约束) xxxxx DROP CONSTRAINT constraint_name(删除约束) (注意NOT NULL约束的特殊性)
修改列数据类型	xxxxx ALTER COLUMN colname TYPE coltype
重命名列表	xxxxx RENAME COLUMN colname TO new_colname ALTER TABLE tablename RENAME TO new_tablename

需要注意的是，以上修改操作并非一定成功。例如，删除某列时如果该列存在外加键约束并被其他表引用，则无法删除该列，需要在语句末尾添加 "CASCADE" 关键字，表示尽可能地级联删除与该列存在关联的数据。

在修改过程中可通过在终端输入 "\d 表名" 查看修改后的表信息。

为SPJ表中SNO,PNO,JNO分别添加外键约束

```
fkey_S = "ALTER TABLE SPJ ADD FOREIGN KEY (sno) REFERENCES S;"
fkey_P = "ALTER TABLE SPJ ADD FOREIGN KEY (pno) REFERENCES P;"
fkey_J = "ALTER TABLE SPJ ADD FOREIGN KEY (jno) REFERENCES J;"

# 执行sql
cur.execute(fkey_S)
cur.execute(fkey_P)
cur.execute(fkey_J)
```

为SPJ增加属性列QTY 数据类型INT，表示工厂向供应商购买某种零件的数量，并添加检查约束 (QTY > 0 且非空)

```
sql = "ALTER TABLE spj ADD COLUMN QTY INT NOT NULL CHECK (QTY > 0);"
cur.execute(sql)
```

为P表WEIGHT属性添加NOT NULL约束。

```
sql = "ALTER TABLE P ALTER COLUMN WEIGHT SET NOT NULL;"
cur.execute(sql)
```

执行下述SQL，插入初始数据

```
insert into S(Sno,Sname,Status,City)
values
('S1','精益','20','天津'),
('S2','盛锡','10','北京'),
('S3','东方红','30','北京'),
('S4','丰泰盛','20','天津'),
('S5','为民','30','上海');

sql = "INSERT INTO S(SNO,SNAME,STATUS,CITY) VALUES ('S1','精益','20','天津'),('S2','盛锡','10','北京'),('S3','东方红','30','北京'),
('S4','丰泰盛','20','天津'),('S5','为民','30','上海');"
cur.execute(sql)
```

分别编写违反 SPJ外键约束、SPJ->QTY检查约束、P->WEIGHT非空约束 的数据，分别查看SQL执行的结果

违反SPJ外键约束 尝试在SPJ表中添加从供应商S6(不存在)进货的数据

```
sql = """insert into SPJ(sno, pno, jno, qty)
      values
      ('S6', 'P1', 'J1', 200);"""
cur.execute(sql)
```

报错信息：

```
ForeignKeyViolation: insert or update on table "spj" violates foreign key constraint "spj_sno_fkey"
DETAIL:  Key (sno)=(S6) is not present in table "s".
```

出现了外键错误。

违反QTY检查约束 尝试添加QTY小于等于0的数据

```
sql = """insert into SPJ(sno, pno, jno, qty)
        values
        ('S1', 'P2', 'J3', -300);"""
cur.execute(sql)
```

报错信息:

```
CheckViolation: new row for relation "spj" violates check constraint "spj_qty_check"
DETAIL:  Failing row contains (S1, P2, J3, -300).
```

检查qty约束时出现错误。

违反P->WEIGHT非空约束 尝试添加WEIGHT为空的数据到P

```
sql = """insert into P(pno, pname, color, weight)
        values
        ('P7','轮轴','白',NULL);"""
cur.execute(sql)
```

报错信息:

```
NotNullViolation: null value in column "weight" of relation "p" violates not-null constraint
DETAIL:  Failing row contains (P7, 轮轴, 白, null).
```

检查not null时出现错误。

查询数据（单表查询）

在终端中执行下述命令，先删除SPJ，以防违反外键约束。

```
DELETE FROM SPJ;
DELETE FROM S;
DELETE FROM P;
DELETE FROM J;
```

找出供应的零件数量不少于500的供应商与零件号的组合

```
sql = "SELECT S.SNAME,SPJ.PNO FROM S,SPJ WHERE S.SNO=SPJ.SNO AND SPJ.QTY>=500"
cur.execute(sql)
for i in cur.fetchall():
    print(i)
```

('精益 ', 'P1')

('盛锡 ', 'P3')

('为民 ', 'P6')

也可以这样:

```
sql = """select sno, pno
        from SPJ
        where QTY >= %s;"""
cur.execute(sql, (500,))
```

找出零件为P1或P2或P5 且 供应商为 S2或S5或S6 的工厂号

```
sql = "SELECT JNO FROM SPJ WHERE (SNO='S2' OR SNO='S5' OR SNO='S6') AND (PNO='P1' OR PNO='P2' OR PNO='P5')"
cur.execute(sql)
```

列出为J4工厂供货的供应商号，要求去重

```
sql = "SELECT DISTINCT SNO FROM SPJ WHERE JNO='J4'"
cur.execute(sql)
```

删除数据表

```
sql = "DROP TABLE SPJ; " + \
      "DROP TABLE S; " + \
      "DROP TABLE P; " + \
      "DROP TABLE J; "
cur.execute(sql)
```

3.常用关键字

```
# 导入 Python 与 PostgreSQL 操作库
import psycopg2
conn = psycopg2.connect(host="172.16.253.154", port="5432", user="ecnu10211900416", password="ECNU10211900416",
database="ecnu10211900416")
autocommit = psycopg2.extensions.ISOLATION_LEVEL_AUTOCOMMIT
conn.set_isolation_level(autocommit)
cur = conn.cursor()
```

创建四张表:

表一、S(SNO,SNAME,STATUS,CITY);

供应商表 S, 供应商代码 (SNO), 供应商姓名 (SNAME), 供应商状态 (STATUS), 供应商所在城市 (CITY)

表二、P(PNO,PNAME,COLOR,WEIGHT);

零件表 P, 零件代码 (PNO), 零件名字 (PNAME), 颜色 (COLOR), 重量 (WEIGHT)

表三、J(JNO,JNAME,CITY);

工程项目表 J, 工程代码 (JNO), 工程名字 (JNAME), 工程所在城市 (CITY)

表四、SPJ(SNO,PNO,JNO,QTY);

供应情况表 SPJ, 供应商代码 (SNO), 零件代码 (PNO), 工程代码 (JNO), 供应数量 (QTY)

如果这三张表已经存在, 需要自己将这这些张表删除

```
#删除数据表
sql = "DROP TABLE SPJ; " + \
      "DROP TABLE S; " + \
      "DROP TABLE P; " + \
      "DROP TABLE J; "

cur.execute(sql)
```

```
sql1="create table S \
(\
Sno char(2) unique,\
Sname char(6),\
Status char(2),\
City char(4),\
primary key(Sno)\
)"

sql2="create table P\
(\
Pno char(2) unique,\
Pname char(6),\
color char(2),\
weight int,\
primary key(Pno)\
);"

sql3="create table J\
(\
Jno char(2) unique,\
Jname char(8),\
CITY char(4),\
primary key(Jno)\
);"

sql4="create table SPJ\
(\
```



```
Sno char(2),\
Pno char(2),\
Jno char(2),\
QTY int,\
primary key(Sno,Pno,Jno),\
foreign key(Sno) references S(Sno),\
foreign key(Pno) references P(Pno),\
foreign key(Jno) references J(Jno)\
);"
cur.execute(sql1)
cur.execute(sql2)
cur.execute(sql3)
cur.execute(sql4)
```

插入数据,确保只运行一次

```
sql1="insert into S(Sno,Sname,Status,City)\
values\
('S1','精益','20','天津'),\
('S2','盛锡','10','北京'),\
('S3','东方红','30','北京'),\
('S4','丰泰盛','20','天津'),\
('S5','为民','30','上海');"

sql2="insert into P(Pno,Pname,color,weight)\
values\
('P1','螺母','红',12),\
('P2','螺栓','绿',17),\
('P3','螺丝刀','蓝',14),\
('P4','螺丝刀','红',14),\
('P5','凸轮','蓝',40),\
('P6','齿轮','红',30);"

.....
```

查看已有的数据表

```
cur.execute("""SELECT table_name FROM information_schema.tables
              WHERE table_schema = 'public'""")

for tuple in cur.fetchall():
    print(tuple)
```

单表查询这种常用关键字

关键字	含义	用法
\$ (not) like	相似匹配	常出现在where中，用来匹配字段中含有某一个特定模式的记录
\$as	字段重命名	用来对select后面的字段重新命名
\$order	排序	常用在一个sql查询的最后面，用来对查询结果做排序
\$limit	限制输出数量	常用在一个sql查询的最后面，用来限制查询结果数量
\$ (not) in	在（一个列表）之内	常出现在where中，用来判断字段值中是否在某个列表之内
\$between . and .	在两数之间	常出现在where中，用来判断字段值中是否在两个值之间
\$distinct	去重	用在select中，用于按照特定字段对结果去重
\$case . when . then	多条件映射	常用在select中，用于将字段中满足条件的值映射成为一个新值

查询零件表P中，零件名含有'螺'字的零件所有信息，并且按照零件重量降序排序

```
sql="select * from P \
where pname like '%螺%' \
order by weight desc"
cur.execute(sql)
#like '%螺%':匹配含有‘螺’的
#like '%螺%':匹配以‘螺’开头的
#like '螺%':匹配以‘螺’结尾的
#desc: 降序
#asc: 升序
```

查询零件表P的所有信息，并且按照根据重量与25的关系来辨别轻重

```
sql="select *, \
case when weight<=25 then '轻' when weight>25 then '重' end as type \
from P \
order by weight desc"
cur.execute(sql)
```

查询零工程项目表J中名字中含有‘厂’字的所有记录，并且需要根据城市名字来判断是不是直辖市

```
sql = "SELECT jno,jname,city, \
CASE WHEN city IN('北京','天津','上海','重庆') THEN '直辖市' ELSE '非直辖市' END AS city_type \
FROM j \
WHERE jname LIKE '%厂%';"
cur.execute(sql)
for tuple in cur.fetchall():
    print(tuple)
```

查询零件表P中质量最轻的前三种零件的信息

```
sql="select * \
from P \
order by weight asc \
limit 3"
cur.execute(sql)
```

4. 聚集函数

类似于mongodb中的聚集函数，postgresql也提供了常用的聚集函数，如max，min，count，avg,sum等。聚集函数的使用常常伴随着分组操作，即按照字段先分组，然后对不同分组的特定字段进行聚合操作。下面来看一个简单的聚合：

零件表P中求不同颜色的零件的平均重量并且保留两位小数

```
sql="select color , round(avg(weight),2) as avg_color \
from P \
group by color"
cur.execute(sql)
```

如果不使用group by的话就是对所有表求均值；一般地，如果select出现了哪些除了聚集之外的字段，那么groupby中也需要对应填上这些字段，不然可能会报错以上，用到了avg聚合函数，和round这个postgresql中内置的字段值转换函数，更多的函数可以参考https://blog.csdn.net/pg_hgdb/article/details/121612991

```
# (1)
sql="select color as c, round(avg(weight),2) as avg_color \
from P \
where color !='绿' \
group by c"
cur.execute(sql)
```

对于以上sql查询，我们需要了解其查询的顺序：

- 1、先做where里面的筛选，即从P表中删除掉颜色为绿色的记录；
- 2、从留存的记录中抽取select中需要用到的列，这里是选出来了color和weight两列；
- 3、根据group by 后面的字段进行分组，这里是按照color进行分组；
- 4、做聚集，化多为一，这里就是将不同颜色组中的重量聚合成他的平均数；
- 5、对聚集之后的字段做一些转换/重命名的操作

了解到where的筛选操作是发生在聚集之前的，那么如果我们想在聚集之后再对其做一些筛选需要怎么做呢？用到having字段！

having字段和where字段都起到筛选作用，但是having发生在where之后，是用来对最终输出结果做一个筛选的。

```
# (2)
sql="select color , round(avg(weight),2) as avg_color \
from P \
group by color \
having color !='绿'"
cur.execute(sql)
```

以上两条查询分别使用了where，having实现了对绿色的剔除，请体会两者的区别

查询供应情况表SPJ,查询不同供应商提供给工程项目（但不包括J1）的零件总数，并且筛选出总供应数大于600的记录

```
sql = "SELECT sno,SUM(qty) \
FROM spj \
WHERE jno!='J1' \
GROUP BY sno \
HAVING SUM(qty)>600;"
cur.execute(sql)
```

5.嵌套查询

嵌套查询指的是两个查询互相嵌套，一个查询作为子查询出现在另一个查询中。子查询常出现在select、from、where这三处。

查询当地没有供应商的工程项目全部信息

```
#子查询出现在where中
sql="select * \
from J \
where CITY not in \
(select distinct City from S) "
cur.execute(sql)
```

剔除绿色求均值

```
#子查询出现在from中
sql="select t.color,round(avg(t.weight),2) as avg_color \
from (select * from P where color !='绿')t \
group by t.color"
cur.execute(sql)
```

例题

```
#子查询出现在select中
sql="select \
(select round(avg(weight),2) as avg_weight from P) as avg_weight, \
(select max(QTY) as max_QTY from SPJ) as max_QTY"
#这种情况往往是用来输出一些独立的信息
cur.execute(sql)
```

当然，如果觉得嵌套查询让一个查询过于繁琐且不直观，我们可以使用with as这个关键字来先创建一个临时表

剔除绿色求均值

```
sql="with tmp as (select color,weight from P where color !='绿') \
select color,round(avg(weight),2) as avg_color \
from tmp \
group by color"
cur.execute(sql)
```

从供应关系表SPJ中找到，供应商和项目工程都不在北京的记录并且按照供应数量升序排序。

提示：with as 创建多张临时表

```
with t as (select * from cx.over_test),
t1 as (select * from t where ord='1')
```

```
sql = "WITH s1 AS (SELECT sno FROM s WHERE city!='北京'), \
j1 AS (SELECT jno FROM j WHERE city!='北京') \
SELECT spj.sno AS sno,pno,spj.jno,qty \
FROM spj,s1,j1 \
WHERE spj.sno=s1.sno AND spj.jno=j1.jno \
ORDER BY qty asc;"
cur.execute(sql)
```

6.多表连接

多表连接查询出现在，我们需要的信息在往往分布在多张表中。来看一个多表查询的例子：

查询供应关系表，并且需要将Sno转换成Sname。

这个虽然通过case--when关键字可以实现，但是编写起来比较繁琐而且不适用于大批量的转换。Sno和Sname对应信息包含在S表中，因此我们需要通过连接两表来查询所需信息。

```
sql="select S.Sname,SPJ.Pno,SPJ.Jno,SPJ.QTY \
from SPJ,S \
where SPJ.Sno=S.sno"
cur.execute(sql)
```

咱们继续来看看这个sql的查询顺序：

1、首先就是执行From中的两张表的全连接，假设这两张表尺寸分别是L1行M列，L2行N列，那么连接后的表尺寸是（L1XL2）行（M+N）列。即表一的每一条记录都会拼接表2的每条记录出现在连接后的表中；

2、对连接后的表执行where筛选；

3、执行select等一些后续操作。

下面再来看一条查询，作用和上一条查询一样：

```
sql="select S.Sname,SPJ.Pno,SPJ.Jno,SPJ.QTY \
from SPJ full join S on SPJ.Sno=S.Sno"
cur.execute(sql)
```

这里使用on这个关键字也实现和where一样的功能，但是这两者的区别是什么呢？where和on的区别？哪一种方式效率高？

mx:

当需要实现多表连接时，数据库都会生成一张临时表。而构建这个临时表的过程，就体现了where和on的区别。

- where筛选条件是在临时表建成之后执行的。数据库会先将需要连接的表根据其笛卡尔积进行连接形成一张新的临时表，其数据量是被连接的所有表的数据量的乘积，然后在这张巨大的临时表的基础上用where条件进行筛选。
- on筛选条件是在建临时表时就执行的，在一开始为临时表添加数据时就会检查筛选条件，只把符合筛选条件的数据表的笛卡尔积加入临时表，然后直接返回临时表。
- 因此on明显具有空间上的优势，但是在full join的执行效率上，在我看来，ON的效率略优于WHERE。因为full join是将两个表所有的数据的笛卡尔积都存下来了，因此ON和WHERE执行的条件判断次数是一样的，而WHERE建表的代价大于ON，因此其效率更低。而对于其他的join来说，比如left join：在对等查询过程中，ON的查询更快；在不对等查询时，WHERE更快。因为如果存在大量不等数据，就会出现大量的空值，这时候临时表查询效率变得更低，而选择WHERE查询能够在查询时自动过滤不等的数据。

full join,inner join,left join和right join的区别：

这是两张数据表连接的三种方式，都需要后面接上on关键字。

对于A inner join B on A.id=B.id:只有A，B都含有的id记录才会出现在连接后的表中，保留了两表的共有字段；

对于A full join B on A.id=B.id:任意一方含有的id记录都会出现在连接后的表中，不含id的表的相关字段会赋上空值，保留了两表的全部字段；

对于A left join B on A.id=B.id:连接后的表的id仅来自于A表中，A表不含的id但B表含有的id是不会出现在结果中的,即连接后的表包含了A表的全部字段。常用在对A表进行字段扩充的场景。

对于A right join B on A.id=B.id:参考left join,这里B表是主角

我：

1. where和join ... on在逻辑意义上是完全等价的。join ... on 指明了更清晰的查询计划，而where对查询计划中各个逻辑的执行顺序没有要求；
2. where筛选条件是在临时表建成之后执行的。数据库会先将需要连接的表根据其笛卡尔积进行连接形成一张新的临时表，其数据量是被连接的所有表的数据量的乘积，然后在这张巨大的临时表的基础上用where条件进行筛选。on筛选条件是在建临时表时就执行的，在一开始为临时表添加数据时就会检查筛选条件，只把符合筛选条件的数据表的笛卡尔积加入临时表，然后直接返回临时表。整体来说，where会生成更多的候选执行计划，这导致需要花费更多的时间用于规划，但可能会选到更优的执行计划，使得具体执行的时间减少；join ... on则与之相反；

3. 对于当代postgresql planner, 上述说法并非完全严谨的, 因为即便人们通过join指示了planner执行计划的细节, 但是planner仍然可能不按照指示, 而是自行生成大量候选执行计划并进行挑选. 但由于执行计划的数目随表的数目指数增长, 因此在执行计划过多的时候, planner采用了一种genetic的启发式算法只评估部分候选执行计划的性能, 使用join能改变某些候选执行计划的优先级, 使得它们更早被考虑到. 比如说full join是将两个表所有的数据的笛卡尔积都存下来了, ON和WHERE执行的条件判断次数一样, 但是WHERE建表的代价大于ON, 因此其效率更低; 然而在left join中, 若为对等查询过程, ON更快, 若为不对等查询, WHERE更快.

full join,inner join,left join和right join的区别:

这是两张数据表连接的三种方式, 都需要后面接上 on 关键字.

对与 A inner join B on A.id=B.id: 只有 A, B 都含有的 id 记录才会出现在连接后的表中, 保留了两表的共有字段;

对与 A full join B on A.id=B.id: 任意一方含有的 id 记录都会出现在连接后的表中, 不含 id 的表的相关字段会赋上空值, 保留了两表的全部字段;

对与 A left join B on A.id=B.id: 连接后的表的 id 仅来自于 A 表中, A 表不含的 id 但 B 表含有的 id 是不会出现在结果中的,即连接后的表包含了 A 表的全部字段. 常用在对 A 表进行字段扩充的场景.

对与 A right join B on A.id=B.id: 参考 left join, 这里 B 表是主角

将S表和SPJ表通过Sno按照不同的方式连接

```
sql="select S.* from S"
cur.execute(sql)
```

('S1', '精益 ', '20', '天津 ')

('S2', '盛锡 ', '10', '北京 ')

('S3', '东方红 ', '30', '北京 ')

('S9', 'name ', '11', '台湾 ')

```
#left join
sql="select S.*, '||||', SPJ.* \
from S left join SPJ on SPJ.Sno=S.sno"
cur.execute(sql)
```

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J1', 200)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J3', 100)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J4', 700)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P2', 'J2', 100)

('S2', '盛锡 ', '10', '北京 ', '||||', 'S2', 'P3', 'J1', 400)

('S2', '盛锡 ', '10', '北京 ', '||||', 'S2', 'P3', 'J2', 200)

('S2', '盛锡 ', '10', '北京 ', '||||', 'S2', 'P3', 'J4', 500)

.....

('S9', 'name ', '11', '台湾 ', '||||', None, None, None, None)

```
#full join
sql="select S.*, '||||', SPJ.* \
from S full join SPJ on SPJ.Sno=S.sno"
cur.execute(sql)
```

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J1', 200)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J3', 100)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J4', 700)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P2', 'J2', 100)

('S2', '盛锡 ', '10', '北京 ', '||||', 'S2', 'P3', 'J1', 400)

.....

('S9', 'name ', '11', '台湾 ', '||||', None, None, None, None)

```
#inner join
sql="select S.*, '||||', SPJ.* \
from S inner join SPJ on SPJ.Sno=S.sno"
cur.execute(sql)
```

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J1', 200)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J3', 100)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P1', 'J4', 700)

('S1', '精益 ', '20', '天津 ', '||||', 'S1', 'P2', 'J2', 100)

.....

观察上面S9的记录，体会这几种join方式的区别

【这里我要网上查阅资料之后总结一下！】

将SPJ表中的no全部转化为name

```
sql="SELECT S.sname,P.pname,J.jname,SPJ.qty \
FROM ((SPJ LEFT JOIN S ON SPJ.sno=S.sno) \
LEFT JOIN P ON SPJ.pno=P.pno) \
LEFT JOIN J ON SPJ.jno=J.jno;"
cur.execute(sql)
```

对零件表P按照color分组，并且求出每一组重量的方差。要求不使用方差或标准差的聚集函数，只使用sum, avg, count等上述提到的聚集函数。

```
sql = "WITH avg_weight AS (SELECT color,AVG(weight) AS avg_w FROM P GROUP BY color) \
SELECT P.color,ROUND(AVG((weight-avg_w)*(weight-avg_w)), 2) AS Var_weight \
FROM P LEFT JOIN avg_weight ON P.color=avg_weight.color \
GROUP BY P.color;"
cur.execute(sql)
```

描述你所编写的练习6中sql的查询顺序

mx:

- 1. 执行WITH AS操作，创建临时表，该步又分以下几步：
 - a. 从P表选出color和weight两列数据
 - b. 将该数据按照color进行分组
 - c. 计算聚合函数AVG()
 - d. 将计算出来的AVG()函数值所在列命名为avg_w
 - e. 将得到的数据表命名为avg_weight
- 2. 执行LEFT JOIN ... ON，对于P和avg_weight中的所有数据的笛卡尔积，检查其P.color=avg_weight.color的真假，若真，则将该数据JOIN到临时表
- 3. 从临时表中选出P.color,P.weight,avg_weight.avg_w 3列需要的数据
- 4. 将选出的数据按照P.color分组
- 5. 计算聚合函数 ROUND(AVG((weight-avg_w)*(weight-avg_w)), 2) 的值。
- 6. 将聚合函数计算结果命名为Var_weight，按照P.color放在对应位置，作为新的一列数据。
- 7. 返回查询结果

我：

- 按颜色分组, 计算每组体重平均值 p_avg.weight, 并存储在临时表 p_avg 中
- 将原表 p 与 p_avg 根据颜色字段拼接, 再根据颜色分组:
- 对于每一组, 计算p.weight与p_avg.weight的离差平方, 再求平均数, 最后保留2位小数.

7.窗口函数

我们查询了P表中所有零件的前三重量。但是如果我们需要知道不同颜色分组前三的质量呢？那就需要用到窗口函数

SPJ表，查询不同供应商供应量前三的信息

```
sql="select *,rank() OVER (PARTITION BY Sno ORDER BY QTY DESC) as group_rank from SPJ"
cur.execute(sql)
```

```
('S1', 'P1', 'J4', 700, 1)
('S1', 'P1', 'J1', 200, 2)
('S1', 'P2', 'J2', 100, 3)
('S1', 'P1', 'J3', 100, 3)
```

.....

从以上结果看出，除了SPJ表中的所有信息之外，结果中还多了一列，这就是窗口函数所起的作用---分组编号。

来看rank() OVER (PARTITION BY Sno ORDER BY QTY DESC)：

rank():代表了如何编号，这里是标上排名。当然也存在max, avg..等常用“编号”函数

PARTITION BY 某个字段:也就是按照这个分组，可省略那就是整张表作为一组。

ORDER BY 每个字段:查询的结果中每个分组是按照这个字段排序后展示的

还没出现结果? 那就将上面的sql作为子查询后筛选group_rank<=3

值得注意的是, 观看S5的分组:

rank()使得窗口函数那一列的结果是: 1-2-2-4

此外:

dense_rank()可以输出: 1-2-2-3

row_number()可以输出: 1-2-3-4

窗口函数在实际企业大规模的数据分析中是非常实用的一类技巧。

对零件表按重量升序排序, 附加一列累和

```
sql = "SELECT *,SUM(weight) OVER (ORDER BY weight ASC) as sum from P;"
cur.execute(sql)
```

我写的:

```
cur.execute('''
    SELECT *, SUM(weight) OVER(ORDER BY weight)
    FROM p;
''')
```

8.查询优化与执行计划

由于postgreSQL真正的查询优化器实现原理复杂且工程量很大, 因此本次实验内容仅包含查询优化模块最基础的内容: 谓词下推、子查询提升、单表扫描、两表连接。

何为查询优化?

可以想象一个现实中的例子: 现在有个任务, 要求从华东师范大学(中北校区)到外滩, 请问该如何设计通行路线使所用时间最少。可以确定的是, 到达外滩的可选路线有无数条, 但是我们不可能都走一遍来确定最优路线。我们首先可以从经验入手推导出设计路线的原则, 比如尽量使总路程最短、尽量减少步行的距离、尽量乘坐通行速度快的交通工具、尽量减少更换交通工具的次数等, 由此可以排除掉大量的路线, 得到若干个备选方案(已经远小于初始的问题空间)。接下来我们可以利用真实的数据, 比如目前上海市中心车流量堵塞情况、步行/共享单车/地铁等通行方式的速度、地铁的等待时间与换乘时间、红绿灯等待时间等数据, 来对不同的备选方案进行时间的估算, 最终可以得到优化的通行路线。

关系数据库系统查询优化的原理与上述过程类似, 当RDBMS接收到一条SQL查询语句, 需要将SQL语句描述的做什么(what)转化为怎么做(how)。这个过程大体可以划分为三个阶段:

1. 将SQL语句转化为语法分析树
2. 利用关系代数等价转换进行逻辑优化生成关系代数表达式
3. 利用系统与统计信息对不同的查询路径进行代价估计进行物理优化, 最终生成查询执行计划。

其中, 第一个阶段本次lab不做探讨, 本次lab简要讲解后两个阶段。

创建新的S, P, J, SPJ关系表, 其中S.Sno、P.Pno、J.Jno、SPJ.SPJ_ID属性为主键, 存在B+树索引。SPJ中的Sno、Pno、Jno属性为外键属性, 没有B+树索引。

```
create table S (Sno int , Sname char(6), City char(4), primary key(Sno));
create table P (Pno int , Pname char(6), weight int, primary key(Pno));
create table J (Jno int , Jname char(6), City char(4), primary key(Jno));
create table SPJ ( SPJ_ID int, Sno int, Pno int, Jno int, QTY int,
                  primary key(SPJ_ID), foreign key(Sno) references S(Sno),
                  foreign key(Pno) references P(Pno), foreign key(Jno) references J(Jno));
```

为了观察到一条SQL查询具体的执行计划, 需要用到explain命令。explain命令是postgresql数据内置的一条命令, 命令格式为:

```
explain [option] [SQL查询语句];
```

```
testdb=# explain select * from p where p.pno = 1;
               QUERY PLAN
-----
Seq Scan on p  (cost=0.00..1.63 rows=1 width=15)
  Filter: (pno = 1)
(2 行记录)
```

输出 "select * from p where p.pno = 1;" 这条语句经过postgreSQL查询优化器后生成的执行计划，但并未真正执行，输出的数值均为优化器预测值。输出结果中每个部分的意义会在后面解释。

如果想实际运行SQL语句，获得真实值，可以加上analyze选项，让SQL语句真正运行：

```
testdb=# explain analyze select * from p where p.pno = 1;
               QUERY PLAN
-----
Seq Scan on p  (cost=0.00..1.63 rows=1 width=15) (actual time=0.523..0.526 rows=1 loops=1)
  Filter: (pno = 1)
  Rows Removed by Filter: 49
Planning Time: 0.736 ms
Execution Time: 0.566 ms
(5 行记录)
```

"actual time"与"Execution Time"是在SQL语句执行过程得到的真实数据。

8.1逻辑优化之谓词下推

现有一个查询：

```
select P.Pno, P.Pname
from SPJ, P
where SPJ.Pno = P.Pno and SPJ.QTY >= 500;
```

该查询可以写成一个关系代数表达式：

$$\pi_{(P.Pno,P.Pname)}(\sigma_{SPJ.QTY \geq 500}(SPJ \bowtie_{spj.pno=p.pno} P))$$

该关系代数表达式表示首先将SPJ与P两表的数据进行join操作生成中间结果，之后筛选出QTY >= 500元组，最后将筛选出的元组中Pno Pname两列输出。

这个关系代数表达式仅为上述SQL查询语义的一种表达方式，实际上还存在许多种表达同一语义的关系代数表达式（有很多条到达外滩的道路）。逻辑优化的目的，就是运用关系运算符与运算规则，对一个初始的关系表达式在保持语义不变的前提下做变换（也称等价变换），降低SQL执行过程中的时间与空间复杂度。

常见的逻辑优化有谓词下推、子查询提升、外链接消除、条件化简等方法。本次lab仅简要简介谓词下推和子查询提升。

首先讲解谓词下推。

选择、投影、连接是SQL查询最基础的三种操作，连接操作涉及物理优化，选择、投影操作的逻辑优化原则如下：

- 选择：选择操作本质是对一组元组集合做限定条件，筛选出符合条件的元组。由于选择操作会显著减少元组的数量，因此应将选择操作尽量下推到更低一级的运算中，以减少生成的中间结果，达到优化I/O和CPU消耗的目的。
- 投影：投影的本质是限制返回元组的列。优化投影操作方式与选择操作类似，尽量下推到更低一级的运算，减少生成的中间元组的大小。与选择操作不同的是，投影操作是消减单个元组的大小，而选择操作是消减元组的数量（一个纵向一个横向）。

上述关系代数表达式中 $\sigma_{SPJ.QTY \geq 500}$ 可以选择谓词下推，在SPJ与P表做连接之前先作用在SPJ表上，在不改变语义的前提下，消减SPJ与P表连接的中间结果数量。根据投影的串接律：如果 $A_1, A_2, \dots, A_n$ 是 $B_1, B_2, \dots, B_m$ 的属性子集，则 $\pi_{A_1, A_2, \dots, A_n}(\pi_{B_1, B_2, \dots, B_m}(E)) = \pi_{A_1, A_2, \dots, A_n}(E)$ ，由于 $\pi_{P.Pno, P.Pname}$ 是P表属性的子集，因此可将投影 $\pi_{P.Pno, P.Pname}$ 下推至P表，以进一步消减SPJ与P表连接生成的中间结果大小。

因此，上述关系表达式在应用选择与投影操作谓词下推优化下，可转化为以下语义等价的关系表达式：

$$\pi_{(P.Pno,P.Pname)}((\sigma_{SPJ.QTY \geq 500}(SPJ)) \bowtie_{spj.pno=p.pno} (\pi_{P.Pno, P.Pname}(P)))$$

显然，相较原关系表达式先连接、再选择、最后投影的操作顺序，优化后的关系表达式将选择、投影下推，先对表SPJ表做选择操作，筛选出符合条件的元组；再对P表做投影减少连接时P表的大小，最后进行连接。显著降低了中间结果的大小，提高了I/O和CPU效率。

8.2逻辑优化之子查询提升

子查询在SQL语句中是一类常见操作。在早期的数据库实现中，查询优化器会采用嵌套执行的方式，即父查询每处理一行，子查询就调用一次。这样带来的结果是，父查询中执行了多少行，子查询就被调用多少次，导致执行效率低下。

因此，对子查询的优化对SQL查询的执行性能会带来很大的影响。

子查询的类型有很多，根据子查询和父查询之间有没有相关性可分为：相关子查询和非相关子查询；根据子查询操作符可划分为：[NOT]IN/ALL/ANY/SOME子查询与[NOT]EXISTS子查询等，不同的子查询类型有不同的优化方式。

本节以in子查询为例，研究子查询优化中的一种关键方法：子查询提升。

```
select *
from J
where J.city in (select city from S);
```

上述SQL查询表达的语义是：找出所在地存在工厂（S）的零件厂（J）。

如果按照早期的执行方式执行上述查询，会对J表中每一条元组，都执行一遍子查询 $select city from S$ ，然后判断J.city是否在子查询返回的结果中。显然子查询会执行|J|遍，造成严重的性能浪费。

现在SQL查询优化器会尽量将子查询“提升”，提升到和父查询同一层次。比如上述的SQL语句，根据执行的流程可知，子查询每次都在做重复的工作，且目的在于提供“city”集合，供父查询中J表元组做匹配操作，如果J.city在集合中则返回，如果不在集合中则跳过。根据前面的分析可知，in操作的语义与join操作类似，在join操作中左连接元组属性值在右连接元组中不存在，则不会连接成连接元组；如果存在，则会生成连接元组，生成连接结果。因此可以将上述SQL中的in操作转化为join操作。

上述SQL经过子查询提升可得优化后的关系代数表达式：

$$\pi(J \bowtie_{J.city=S.city} \pi_{S.City}(S))$$

8.3 物理优化之单表扫描

通过基于关系代数等价运算规则进行逻辑优化，关系数据库可以得到一个优化后的关系代数表达式，一定程度决定数据库该以什么顺序执行什么样的操作，将SQL查询语言从“做什么（what）”向“怎么做（how）”迈出了一大步。数据库从关系代数表达式中虽然可以知道该以什么顺序做选择、投影，但是其依然不清楚具体的每一步该怎么执行，比如对单个表是采用顺序遍历还是索引遍历？两表连接是采用hash连接还是嵌套循环连接？以及多张表的连接顺序该如何确定？

因此数据库系统需要对关系代数表达式做进一步处理，利用页面I/O花费、元组CPU执行时间、表的物理信息（页面数量、元组数量、属性上是否有索引）等信息，对不同的物理执行方式做代价评估，最终选出一条代价最小的物理执行路径，这条路径不仅包含操作的顺序，也指明了实现操作的方式。

由于单表扫描是复合操作的基础（可以理解为查询执行树的叶节点），是一个查询执行计划的基础组成成分，因此我们先对单表扫描操作进行学习研究。

首先利用explain命令执行SQL：

```
testdb=# explain select Jno, Jname from J;
               QUERY PLAN
-----
Seq Scan on j  (cost=0.00..78.00 rows=5000 width=11)
(1 行记录)
```

在explain命令作用下"select Jno, Jname from J"命令并未真正执行，而是输出查询执行计划(QUERY PLAN)。现在对输出的执行计划每个数据项进行逐一介绍：

- **Seq Scan on j**: 表示对J表进行顺序扫描，即从表的第一个页的第一个元组扫描到最后一个元组。由于没有选择条件，数据库查询优化器对 $\pi_{Pno, Pname}(J)$ 选择了顺序扫描的物理执行方式。
- **cost=0.00..78.00**: 表示该条执行计划的代价估算，第一个'0.00'表示执行计划的启动成本；'78.00'表示从启动到完成的总代价估计，一般有三部分代价构成：启动代价 + I/O代价 + CPU时间代价。
- **rows=5000 width=11**: 表示此计划节点输出的估计总行数，以及输出元组的估计平均宽度。

执行下述命令，可以看到去掉投影操作后，虽然代价估计值与输出行数没有变化，但是输出的元组宽度却增加到16。如果对J表的扫描操作是其他复合操作（例如上文关系代数中的连接操作）的基础操作，会导致中间结果膨胀，可能会造成更多内存消耗，增加I/O。

```
testdb=# explain select * from J;
               QUERY PLAN
-----
Seq Scan on j  (cost=0.00..78.00 rows=5000 width=16)
(1 行记录)
```

对查询添加限定条件"J.jno > 4950"，检索Jno大于4950的记录。由于J.jno列存在B+树索引，且元组按照Jno大小顺序存放，因此查询优化引擎选择Index Scan（索引扫描的方式）检索数据。索引扫描的原理是，利用索引根据查找键快速定位到元组的位置，可以消减不必要的元组扫描，但同时会带来索引扫描与随机I/O的额外开销。根据QUERY PLAN中代价估算值cost可知，根据选择条件并使用索引时执行 $\sigma_{J.Jno > 4950}(J)$ 的估计cost为9.16，相较表的顺序扫描（78.00 + 每个元组执行操作J.jno > 4950的代价）大大降低了代价。

```
testdb=# explain select * from J where J.jno > 4950;
               QUERY PLAN
-----
Index Scan using j_pkey on j  (cost=0.28..9.16 rows=50 width=16)
   Index Cond: (jno > 4950)
(2 行记录)
```

经过上述三个例子可以大致得到结论：完成单表扫描获取符合条件的元组实际存在多种执行方式，具体选择何种物理执行方式，postgresql查询优化器会根据能否使用索引，I/O与CPU花费代价cost等标准对不同的物理执行方式进行代价评估，选择代价最小的物理执行方式，作为SQL查询的物理执行代价。

接下来我们简要介绍postgresql代价估算的方法。

PostgreSQL查询优化代价估算基于CPU开销和I/O开销，其计算公式如下：

$$\text{总代价} = \text{启动代价} + \text{IO代价} + \text{CPU代价}$$

为了提供较为准确的代价评估，postgreSQL在系统中内置了多种代价常数，其中基础的5个参数如下所示：

- **seq_page_cost = 1**：表示从磁盘顺序读入页面的代价
- **random_page_cost = 4**：表示从磁盘随机读入页面的代价
- **cpu_tuple_cost = 0.01**：cpu处理每个元组的代价
- **cpu_index_tuple_cost = 0.005**：在索引扫描期间CPU处理每个索引条目的代价
- **cpu_operator_cost = 0.0025**：CPU在查询处理期间执行每个操作符或函数的代价

例如上述第1条与第2条SQL语句，查询优化器选择对表j进行顺序扫描（从磁盘中顺序读入页面），之后CPU读取每一条元组，且没有任何判断条件。因此，对表的顺序扫描代价计算公式为：

$$Cost(seq_scan(J)) = relpages * seq_page_cost + reltuples * cpu_tuple_cost$$

我们通过运行下述SQL命令获取表j的页面和元组数量(注意"j"要小写)：

```
testdb=# SELECT relpages, reltuples FROM pg_class WHERE relname = 'j';
 relpages | reltuples 
-----+-----
      28 |      5000
```

带入代价计算公式可得：28 * 1 + 5000 * 0.01 = 78.00，和输出的查询计划代价计算结果相同。

输入下述命令，当判断条件改为j.jno > 0时，postgreSQL优化器选择对表j进行顺序扫描（seq scan）从第一条元组顺序遍历到最后一条元组。下述语句相较第2条语句都采用顺序扫描，不同之处在于对每条元组多出来一次判断处理（j.jno > 0）。因此，代价计算公式为：

$$Cost(seq_scan(J)) = relpages * seq_page_cost + reltuples * cpu_tuple_cost + reltuples * cpu_operator_cost$$

带入代价计算公式可得：28 * 1 + 5000 * 0.01 + 5000 * 0.0025 = 90.50，和输出的查询计划代价计算结果相同。

```
testdb=# explain select * from j where j.jno > 0;
               QUERY PLAN
-----
Seq Scan on j  (cost=0.00..90.50 rows=5000 width=16)
  Filter: (jno > 0)
(2 行记录)
```

针对第3条SQL语句中的索引扫描，其查询计划代价评估更为复杂一些。首先是不同的索引类型，有着不同的代价评估函数；其次，数据的分布也会影响到索引扫描的效率（顺序分布、随机分布、正态分布等）。不过不管是哪种情况，索引扫描也要遵循“总代价 = IO代价 + CPU代价”的基本原理。本次lab针对索引扫描代价评估仅讨论上述5个参数构成的代价模型，不讨论更为复杂的针对索引性能的估计，仅关注索引扫描和顺序扫描代价模型主要的异同。

索引扫描的代价模型为：

$$Cost(IndexScan(j)) = x1 * seq_page_cost + x2 * cpu_tuple_cost + x3 * cpu_operator_cost + x4 * random_page_cost + x5 * cpu_index_tuple_cost$$

由于再搜索符合条件(j.jno > 4950)时使用了索引，因此并不需要像顺序扫描遍历全部的元组，可以利用索引定位一个范围，大大消减遍历的元组的数量，因此x1 x2 x3三个系数相较顺序扫描减少。但是索引扫描又会带来新的代价，对索引进行搜索时，需要访问索引页面内的索引记录，因此带来 $x5 * cpu_index_tuple_cost$ 的开销；由于postgreSQL中索引页面与数据页面不在同一个数据文件中，因此当从索引叶节点访问对应的数据页面时，需要一次随机页面访问（随机I/O），对于磁盘来说，随机I/O的成本要远大于顺序I/O，因此带来 $x4 * random_page_cost$ 的额外开销。

第3条SQL语句的代价估计为9.16（阿里云数据库上的代价估计为3.36，可能收到了硬件环境的影响），由于索引扫描代价评估涉及到的参数过多以及关键信息的缺失，暂时无法判断代价模型中各参数的具体数值。个人猜测，至少包含一次随机页面访问的代价。

通过上述4个示例展示经过物理优化的单表扫描查询计划，我们可以有一个初步的认识：物理优化是在逻辑优化的基础上，通过获取系统数据（表页面的数量、元组的数量、是否有索引、查询条件等），选择合适的物理执行方式。通过对前三条SQL查询执行计划、执行评估进行分析，第3条SQL相较第2条SQL多了选择条件，减少了返回的元组数量（预测返回的rows从5000减为50）；第1条SQL相较第2条SQL多了映射条件，减少了返回元组的大小（预测的width由16减为11），证明了逻辑优化阶段选择与投影下推可以有效地降低生成中间结果的时间、空间复杂度，提升SQL效率。

8.4物理优化之两表连接

关系代数中重要的操作有选择、投影与连接三部分。上一节讲了选择、投影对单表扫描物理执行计划的影响，本节讲解两表连接运算的物理执行方式。

postgreSQL中包含三种两表连接运算方式，分别是：嵌套循环连接（nested loop join）；排序归并连接（mergejoin）；hash连接（hash join）。其中嵌套循环连接与hash连接操作可以观看水杉课程第六章6.6-6.8节的视频，下面简要讲解排序归并连接。

归并排序连接简称为归并连接，其算法执行流程和归并排序有相似之处。归并连接在做两表连接操作之前，要求两个表分别根据连接属性进行排序，排序后采用类似归并排序中对两个有序数组合并的过程，对两个有序表进行连接操作。具体流程如下：

- 首先以目标SQL中指定的谓词条件（如果有的话）去访问表T1，然后对访问结果按照表T1中的连接列来排序，排好序后的临时表我们记为S1。
- 接着以目标SQL中指定的谓词条件（如果有的话）去访问表T2，然后对访问结果按照表T2中的连接列来排序，排好序后的临时表我们记为S2。

- 最后对结果集1和结果集2执行合并操作：首先有两个指针P1 P2分别指向两个中最小的元组，如果相等则做连接操作，且将其中一个指针向后移动一个元组；如果不相等，例如P1 < P2，则将较小的元组的指针P1向后移动一个元组，之后继续做比较。重复上述操作，直到其中一张表遍历全部的元组，结束连接操作。

在归并连接的过程中，如果连接的列上存在索引，则可利用索引对连接列进行排序，可以降低排序造成的时间花费。

下面展示几个SQL语句使用不同连接算法的例子并解释他们实际的执行流程：

注：spj.qty属性均匀分布在[1, 500] 整数域中。

输入下述命令：

```
testdb=# explain select P.Pno, P.Pname from P, SPJ where SPJ.Pno = P.Pno and SPJ.qty > 300;
               QUERY PLAN
-----
Hash Join  (cost=2.13..2004.06 rows=40365 width=11)
  Hash Cond: (spj.pno = p.pno)
    -> Seq Scan on spj  (cost=0.00..1887.00 rows=40365 width=4)
        Filter: (qty > 300)
    -> Hash  (cost=1.50..1.50 rows=50 width=11)
        -> Seq Scan on p  (cost=0.00..1.50 rows=50 width=11)

(6 行记录)
```

根据explain命令输出的QUERY PLAN可知，上述SQL命令执行流程如下：

- 首先对p表进行顺序扫描，并根据Pno值进行hash，构造出hash桶。（该步骤由于没有选择条件，索引预估输出的元组数量为p表元组总数量50，并且由于符合投影串接率，将pno pname投影下推到此步骤，因此生成的中间元组大小只有11，减少了中间结果大小）

```
-> Hash  (cost=1.50..1.50 rows=50 width=11)
    -> Seq Scan on p  (cost=0.00..1.50 rows=50 width=11)
```

- 接下来对spj表进行顺序扫描，并筛选出 qty > 300的元组。（预估符合条件的元组数量是40365条，宽度为4，但是SPJ表的总宽度远大于4，说明SQL经过优化，此步骤做了投影操作只使用了spj.pno列数据，其余列均未包含在中间结果中，大大降低了中间结果大小）

```
-> Seq Scan on spj  (cost=0.00..1887.00 rows=40365 width=4)
    Filter: (qty > 300)
```

- 最后将第2步得到的符合条件的元组，利用hash join操作，对每个元组进行hash，根据hash值与第1步对应的hash桶内的元组进行匹配，匹配条件为(spj.pno = p.pno)。（由于pno是p表的主键且唯一，因此对于SPJ表中任一元组的pno号，在p表中有且仅有一条元组匹配。因此预估的输出结果为40365条元组，等于符合筛选条件的SPJ表元组数量）

```
Hash Join  (cost=2.13..2004.06 rows=40365 width=11)
  Hash Cond: (spj.pno = p.pno)
```

再测试另一个SQL查询：

```
testdb=# explain select P.Pno, P.Pname from P, SPJ where SPJ.Pno = P.Pno and SPJ.qty > 499;
               QUERY PLAN
-----
Nested Loop  (cost=0.15..1902.56 rows=188 width=11)
  -> Seq Scan on spj  (cost=0.00..1887.00 rows=188 width=4)
      Filter: (qty > 499)
  -> Memoize  (cost=0.15..0.23 rows=1 width=11)
      Cache Key: spj.pno
      Cache Mode: logical
      -> Index Scan using p_pkey on p  (cost=0.14..0.22 rows=1 width=11)
          Index Cond: (pno = spj.pno)
```

根据explain命令输出的QUERY PLAN可知，上述SQL命令执行方式如下：

- 最外层是嵌套循环连接（Nested Loop），表明SPJ与P表使用的连接算法是嵌套循环连接

```
Nested Loop  (cost=0.15..1902.56 rows=188 width=11)
```

- 嵌套循环中的外层循环是对SPJ的单表扫描，采用的是顺序扫描，扫描的同时应用选择条件（spj.qty > 499）。（由于spj表在qty属性上无索引，因此选择顺序扫描，且将选择条件下推到顺序扫描的过程中，可消减参与连接操作的元组数量。预估符合选择条件的元组数量为188）

```
-> Seq Scan on spj (cost=0.00..1887.00 rows=188 width=4)
    Filter: (qty > 499)
```

3. 将外层循环中得到的参与连接的元组，与内层表中符合条件的元组进行连接。根据最下方的执行计划可知，根据spj中符合条件的元组的spj.pno，利用索引对p表进行索引扫描，快速的获取p.pno = spj.pno的元组，之后将二者进行连接操作输出结果。除了对表p的索引扫描，执行计划中还有Memoize物理操作，这个操作是用于嵌套循环中对内层表访问的优化。其作用是，对映射关系 $y = f(x)$ ，Memoize缓存x y的对应关系，因此对到来的x，不再通过f(x)计算得到y，而是直接返回y。在本示例中，x即为pno值，f(x)即索引扫描得到符合（pno = spj.pno）条件的元组这一过程，y即为p中符合条件的元组。如果没有memoize操作，每次对p表找到符合条件的元组都需要进行索引扫描，需要花费额外的时间；如果利用memoize操作，可以将pno与p元组的映射关系缓存下来，对某pno值除了第一次做索引扫描外，后续相同的pno值均可通过缓存的映射关系直接返回对应的元组，可减少索引扫描的开销。

```
-> Memoize (cost=0.15..0.23 rows=1 width=11)
    Cache Key: spj.pno
    Cache Mode: logical
-> Index Scan using p_pkey on p (cost=0.14..0.22 rows=1 width=11)
    Index Cond: (pno = spj.pno)
```

接下来介绍两表连接操作的代价估计模型。

由于两表连接算法的实际代价估算模型非常复杂，会受到算法具体实现方式、硬件环境、数据的分布（顺序、随机或者符合某个概率模型）、数据选择率等因素的影响。因此下面从算法执行流程分析，建立一个抽象的代价估算模型，忽略掉具体的代价细节，下述代价模型均假设S为左连接表，P为右连接表。

1. 嵌套循环连接

$Cost(nested\ loop\ join(S, P)) = Scan(S) + Scan(P) * Select(S)$

2. 归并连接

$Cost(merge\ join(S, P)) = Sort(S) + Sort(P) + Scan(S_tmp) + Scan(P_tmp)$

3. Hash连接

$Cost(hash\ join(S, P)) = Createhash(P) + Scan(S) + Select(S) * Number(P) * 选择率$

Scan表示对表进行扫描操作的代价。不同的物理执行方式有不同的代价，例如嵌套循环连接中，Scan(P)可以采用索引扫描的方式进行优化，并且可以采用Memoize缓存映射关系进行优化。但是对于归并连接，Scan是对S P表排序后的临时表进行扫描，由于有序且无索引，因此采用顺序扫描的方式。

Select表示某张表实际参与连接操作的元组数量。

Sort表示对表进行排序操作的代价。如果排序的列上存在有序索引，则可利用索引进行排序，可降低Sort的代价。

Createhash表示根据表中某个属性创建hash桶的代价。一般来说，P表的元组数量不变且属性分布不变，其创建hash桶的代价也不变。

$Scan(S) * Number(P) * 选择率$ 表示将表S与构造Hash桶后的表P进行hash连接的代价。选择率表示对于表P某个属性的某个取值，对应元组占表P所有元组的比例。如果对表S存在选择、投影等操作且选择列具有索引，可以在Scan(S)阶段采用索引扫描以减少代价。

8.5练习题

习题1 请在保证语义等价的前提下，利用谓词下推、子查询提升对下述SQL语句进行逻辑优化，写出优化后的关系代数表达式。

```
select
  J.Jname, J.City
from
  J, SPJ
where
  J.Jno = SPJ.Jno
and SPJ.qty > 400
and J.city in
  (select city
   from S
   where S.Sname = '东方');
```

mx:

$$\pi_{(J.Jname, J.city)} \left(\left(\pi_{(J.Jname, J.city, J.Jno)}(J) \bowtie_{J.Jno=SPJ.Jno} \sigma_{SPJ.qty > 400}(SPJ) \right) \bowtie_{J.city=S.city} \sigma_{S.Sname='东方'}(S) \right)$$
$$\pi_{(J.Jname, J.city)} \left(\left(\pi_{(J.Jname, J.city, J.Jno)}(J) \bowtie_{J.Jno=SPJ.Jno} \sigma_{SPJ.qty > 400}(SPJ) \right) \bowtie_{J.city=S.city} \sigma_{S.Sname='东方'}(S) \right)$$

习题2 请分别运行以下两条命令：

```
explain select * from SPJ full outer join P on SPJ.Jno = P.Pno and SPJ.QTY > 400 and P.weight > 500;
explain select * from SPJ full outer join P on SPJ.Jno = P.Pno where SPJ.QTY > 400 and P.weight > 500;
```

请根据输出的查询计划，解释当存在对连接表的选择谓词时，where 和 on操作的区别

mxc:两条命令运行结果分别如下：

```
QUERY PLAN
-----
Hash Full Join (cost=38.58..1938.09 rows=100000 width=56)
  Hash Cond: (spj.jno = p.pno)
  Join Filter: ((spj.qty > 400) AND (p.weight > 500))
  -> Seq Scan on spj (cost=0.00..1637.00 rows=100000 width=20)
  -> Hash (cost=22.70..22.70 rows=1270 width=36)
      -> Seq Scan on p (cost=0.00..22.70 rows=1270 width=36)
(6 rows)
```

```
QUERY PLAN
-----
Hash Join (cost=31.16..1970.93 rows=1699 width=56)
  Hash Cond: (spj.jno = p.pno)
  -> Seq Scan on spj (cost=0.00..1887.00 rows=20086 width=20)
      Filter: (qty > 400)
  -> Hash (cost=25.88..25.88 rows=423 width=36)
      -> Seq Scan on p (cost=0.00..25.88 rows=423 width=36)
          Filter: (weight > 500)
(7 rows)
```

两者的SQL指令存在的差异：前者的SQL指令用了ON作为filter，而后者使用WHERE筛选再连接。

两条explain指令的运行结果也存在差异。

- 对于前者只使用ON作为filter，所有ON的筛选条件都放在了Join Filter里，而scan和hash操作都放在了Join操作下。也就是扫描和连接的同时就在执行Join操作的筛选。
- 而对于后者来说，Filter作用于扫描的过程，也就是先对扫描的结果进行筛选，然后再进行哈希，最后返回连接后的结果。

两者的本质差异在于，ON的条件判断执行在内部循环的每一条数据，也就是说每生成一条连接数据，都会用Join Filter判断是否符合条件，如果符合条件，则加入临时表。而WHERE的条件判断执行在每个表的扫描之后，对于每个表的筛选后数据进行连接，此时就是全连接了，不需要再判断条件。

习题3 在“物理优化之单表扫描”一节中，第3条和第4条查询语句仅改变了判断条件（j.jno > 4950; j.jno > 0），SQL优化器却选择了不同的单表扫描方式。可以尝试下述操作寻找规律：

```
explain select * from j where j.jno > n;
```

n由5000逐渐降低（在本地postgresql14运行时在n=3210附近；在阿里云数据库环境运行时在n=1678附近），对表的扫描操作由索引扫描变为了顺序扫描。

根据第5节中对顺序扫描、索引扫描执行过程以及代价估计的分析，请简要分析出现上述现象的原因。（不需要具体的数值计算）

mxc:测试结果：

```
stu10205501417=> explain select * from j where j.jno > 1679;
               QUERY PLAN
               -----
Index Scan using j_pkey on j (cost=0.28..89.60 rows=3321 width=16)
  Index Cond: (jno > 1679)
(2 rows)

stu10205501417=> explain select * from j where j.jno > 1678;
               QUERY PLAN
               -----
Seq Scan on j (cost=0.00..90.50 rows=3322 width=16)
  Filter: (jno > 1678)
(2 rows)
```

由于postgresql中索引页面与数据页面不在同一个数据文件中，因此当从索引叶节点访问对应的数据页面时，需要一次随机页面访问（随机I/O），对于磁盘来说，随机I/O的成本要远大于顺序I/O，因此，当满足条件的元组数量超过一定上界时，其随机I/O产生的额外代价可能会抵消掉索引对搜索的加速作用，因此索引搜索在这上面存在一个双刃剑的作用。所以，当n较大时，满足条件的元组较少，随机I/O产生的代价较小，数据库系统选择使用索引；而n较小时，满足条件的元组较多，此时再使用索引的话，多次的随机I/O会使效率大打折扣，因此数据库系统会选择搜索更慢但是访问数据更快的顺序检索。

习题4：

在“物理操作之两表连接”一节中，测试的两条SQL语句仅改变了选择条件（qty > n），但是却选择了不同的join算法。

当n不断增加时，spj表符合条件的元组越来越少，则参与连接的元组也相应减少，右连接表P参与连接的元组不变。

尝试根据嵌套循环连接与Hash连接执行过程与代价估计模型，简要分析当n增加时，hash join算法和nested loop连接算法的代价估计模型变化情况以及出现上述现象的原因。（不需要具体的数值计算）

mxc: 嵌套循环连接

$Cost(nested\ loop\ join(SPJ, P)) = Scan(SPJ) + Scan(P) * Select(SPJ)$

Hash连接

$Cost(hash\ join(SPJ, P)) = Createhash(P) + Scan(SPJ) + Select(SPJ) * Number(P) * 选择率$

随着n的不断增加：

- 嵌套循环连接的Scan(SPJ)和Scan(P)都没有改变，都是扫描全表，但是Select(SPJ)变小了，因此嵌套循环连接的时间会变少，变少的程度取决于Scan(P)。
- Hash连接的Createhash(P)、Scan(SPJ)、Number(P)和选择率都没有改变，但是Select(SPJ)变小了，因此Hash连接的时间会变少，变少的程度取决于Number(P) * 选择率。

当n变大时，两者时间消耗的乘积项逐渐变小，因此对于嵌套循环连接来说，Scan(SPJ)会成为主要耗时项，而对于Hash连接来说，Createhash(P)和Scan(SPJ)会成为主要耗时项，因此从某种程度上来说Hash连接的时间消耗会更大。但是当n较小时，由于 $Select(SPJ) * Number(P) * 选择率 < Scan(P) * Select(SPJ)$ ，并且这个效率优势在考虑Hash连接额外的Createhash(P)操作耗时之后依然存在，因此n较小时系统自动选择Hash连接，而n较大时，Hash连接的优势逐渐不复存在，而其Createhash(P)的代价被放大了，因此系统自动选择更优的嵌套循环连接。

9.事务处理

PostgreSQL如何处理并发执行的事务，保证事务运行过程中的隔离性与数据一致性？PostgreSQL事务处理模块具体的原理与实现十分复杂，本次实验内容仅包含PostgreSQL多版本并发控制与快照机制部分最基础的原理与实现。

9.1事务的ACID属性与隔离级别

事务的ACID属性：

- 原子性（Atomicity）：一个事务中的操作要么全部都做，要么全都不做
- 一致性（Consistency）：一个事务将数据库从一个一致状态转化为另一个一致状态（满足各种约束，保证数据一致）
- 隔离性（Isolation）：在并发环境下，事务与其他并发的事务运行时是相互隔离的，并发的事务之间不会相互影响
- 持久性（Durability）：事务提交之后，对数据库中状态的变更是永久性的

通过一个例子展示事务的ACID属性：假设有一个事务，包含银行转账逻辑：A B两个账户分别各有1000元，现需要从A账户转100元给B账户，该事务可抽象为下述操作序列

1. 从A账户里减去100元
2. 为B账户里增加100元

先考虑事务的原子性。如果事务仅执行了第一步操作后，系统由于各种原因导致宕机，重新启动后发现了A账户为900，B账户为1000。为了维护事务的原子性，数据库系统需要将“A账户减去100元”这个操作回滚，恢复到事务执行前的状态，就好像转账事务未发生一样。也就是说数据库系统要保证一个事务中的操作，要么全部都做，要么全都不做，保证事务的原子性。

再考虑事务的隔离性。如果在转账事务执行完第一步还未执行第二步时，有一个并发的读取事务分别读取A、B账户的余额，发现此时A为900、B为1000，AB账户的总存款凭空少了100元。显然读取事务读到了不一致的数据，主要原因是并发的转账事务影响到了读取事务的执行，违背了事务的隔离性。

上述例子中，读取事务读到了不一致的数据状态，也违反了事务的一致性。事务的一致性也有另一种解释。数据库中存在对数据一致的各种约束，例如账户余额不能少于0元；如果某次转账会导致转出的账户余额为负，则会触发约束条件，因此事务无法提交，维护事务的一致性。

事务持久性是指，事务提交后，对数据的修改不会由于断电、宕机的意外事故导致丢失。目前传统的OLTP数据库系统通常是依靠WAL（预写式日志），通过将事务的日志持久化到磁盘等非易失的存储设备中，以保证事务的持久性。

总的来说，数据库系统需要并发控制机制与基于日志的故障恢复技术来维护事务的ACID属性。本次实验主要探讨postgreSQL的多版本并发控制技术，日志与故障恢复技术不在本次lab的范围内。

9.2事务的隔离级别

通常情况下，在一些典型的OLTP应用场景中，事务的原子性、一致性与持久性是不能违背的，否则可能会出现严重的数据异常且难以恢复。不过，不同的应用场景下对事务的隔离性可以做一定的“放松”。

ANSI SQL92标准定义了数据异常的概念：脏读（dirty read）、不可重复读（unrepeatable read）、幻读（phantom read）；并根据禁止数据异常的能力定义了不同的隔离级别：读未提交（read uncommitted）、读已提交（read committed）、可重复读（repeatable read）与可串行化（serializable）。

1.读未提交

顾名思义，读未提交隔离级别表示，当事务读取一个数据对象时，不用考虑创建该数据对象的事务的状态，直接读取即可。在该隔离级别下，并发事务执行并不受限制，因此事务执行效率很高，但是会出现脏读、不可重复读、幻读的异常现象。

脏读（dirty read）示例：

T1	T2
begin	begin
W(A)	R(A)
abort	commit

上述两个并发执行的事务T1、T2，T1修改元素A后，T2读到了T1修改后A的值。但是由于事务T1由于某种原因导致了abort（没有提交），因此需要将A的值回滚为修改之前的值，这就导致T2读到了一个实际上并不存在的值。这就是 **脏读** 数据异常现象。

2.读已提交

为了避免脏读数据异常，“读已提交”隔离级别对读取的数据做了一定的限制：只能读取已经提交的数据。这样不会出现读取一个并不存在的值，避免出现“脏读”异常。但是“读已提交”依然会出现不可重复读与幻读的异常现象。

不可重复读（phantom read）示例：

T1	T2
begin	begin
R(A)	W(A)
	commit
R(A)	
commit	

如示例所示。T1先读取了A的旧值，T2写入了A的新值并提交，之后T1再次读取A的值，此时读取的是T2写入新值。该执行序列满足“读已提交”隔离级别，但是对于T1来说，前后读取同一个数据对象A得到的值却不同，出现了 **不可重复读** 数据异常。

“读已提交”由于可以保证最基本的隔离性能够满足多数应用的需求，并且能提供较高的并发度，因此是多数商业数据库系统默认的隔离级别。

3.可重复读

“可重复读”隔离级别保证整个事务执行期间，前后读取同一个数据对象的值均相同。但是“可重复读”隔离级别会出现“幻读”异常。

幻读（phantom read）示例：

T1	T2
begin	begin
R(condition)	W(condition)
	commit
R(conditon)	
commit	

如示例所示，T1首先根据where查询条件，读取复合条件的元组；然后，事务T2插入新的元组或者更新旧的元组，这些元组满足T1查询的where条件；T1后续以相同的谓词条件读取数据，会发现读出了新的数据（假设T2插入了复合谓词条件的数据）。对于T1来说，前后两次使用相同的谓词条件，但是读取到的数据却并不相同，因此产生了 **幻读** 异常。

tip：不可重复读与幻读的异同：

首先，这两种异常对于事务T1而言，都是先读取了数据，之后因事务T2“写”数据，导致T1再次读数据时前后两次数据不一样，发生了异常。但是，不可重复读异常针对的是一个 **存在的确定的一个数据对象**，T1前后两次读取该数据对象都存在，T2的写操作改变了已经存在的数据对象的值，导致T1读取该对象的值不相同；而幻读异常针对的是一个 **满足谓词条件的数据集合**，结果集并不确定，T2的“写”操作可以是插入一个新的数据对象，导致T1根据前后两次读取了不同的数据集合。

4.可串行化

可串行化是最高的隔离级别。在可串行化隔离级别下，事务并发执行后的数据状态，等价于将事务按照某个串行执行顺序执行的结果。事务在可串行化调度下，可以避免脏读、不可重复读、幻读数据异常。

总结一下，ANSI SQL92标准中隔离级别与数据异常的关系如下表所示：

隔离级别	脏读	不可重复读	幻读
读未提交	可能	可能	可能

隔离级别	脏读	不可重复读	幻读
读已提交	不可能	可能	可能
可重复读	不可能	不可能	可能
可串行化	不可能	不可能	不可能

9.3多版本并发控制理论基础

为了满足事务的隔离性，避免并发事务在执行的过程中出现数据异常现象，数据库系统需要一套并发控制协议以对事务访问数据做一定的限制。 在水杉平台“当代数据管理系统”课程第八章“数据正确性与事务处理”的课程中，讲解了两阶段封锁协议是如何处理并发事务间的访问数据冲突。 两阶段封锁协议通过对需要访问的数据对象加锁，并将整个事务划分为加锁阶段和放锁阶段，以获得访问数据对象的权限，限制并发事务在事务粒度上访问同一个数据对象。

在两阶段封锁协议下，当并发的两个事务存在读写冲突时（访问同一个数据对象），由于封锁协议，一个事务的读（写）操作需要等待另一个事务的写（读）锁释放后，才可以继续执行。 对于只读事务（不存在写操作），访问数据访问广、持锁时间较长，会频繁的与并发的写事务出现冲突，导致严重的性能下降。

多版本并发控制协议有效地解决了读写并发、只读事务优化的问题。 多版本并发控制举例：

```
T1(TS(T1)=1)    T2(TS(T2)=2)
begin
                begin
    R(A)          W(A)

    R(A)
commit          commit
```

数据对象A的初始版本为:

version	data	begin	end
A ₀	10	0	-

事务T1开始时，系统会为其分配一个时间戳TS(T1)=1。在第一次读取A时，遍历到数据版本A₀，其开始时间戳begin=0，结束时间戳end=无穷，表示A₀的“可见时间”为 0 ~ 无穷。显然T1的时间戳落在了A₀的“可见范围内”，因此T1第一次读取A读到了 A₀。

后续T2开始时，系统为其分配一个时间戳TS(T2)=2。当T2写A时，由于采用多版本并发控制协议，T2并不会直接将A的旧值覆盖掉，而是创建一个新的版本A₁，保留A₀并将其end时间戳置为2，表示A₀ 版本对于时间戳大于2的事务不可见。

T2执行W(A)后数据库状态如下：

version	data	begin	end
A ₀	10	0	2
A ₁	20	2	-

当T1第二次读取R(A)时，依然根据时间戳TS(T1)=1遍历A的版本链，发现版本A₀的begin - end时间戳范围包含T1的时间戳，因此A₀对事务T1可见。 T1第二次读取A时，读到的依然是A₀，T2创建的新版本A₁对T1不可见。

通过上述多版本并发控制示例可以看到，多版本并发控制协议会给事务分配一个用于元组版本可见性判断数据结构（在上述示例中是“时间戳”，在PostgreSQL中是快照，下文会讲）。事务根据这个“时间戳”（postgresql中是快照），会读取对自己“可见”的元组版本；当更新元组时，不会在旧版本上直接覆盖新值，而是创建一个新版本，并对旧版本的时间戳进行修改，标识版本的“可见范围”。

通过多版本并发控制协议，并发的两个事务对同一个数据对象 **读写互不阻塞**，相较两阶段封锁协议提高了并发度。 由于其 **读写互不阻塞** 的优秀特性，目前很多商业数据库已经支持了多版本并发控制，例如postgresql、mysql、oracle等。 接下来内容，将对PostgreSQL数据库多版本并发控制原理进行讲解。

PostgreSQL并发控制基础数据结构与操作

经过刚刚那段对多版本并发控制协议基础理论的讲解，可以有初步的认识：

- 事务需要有一个数据结构，标识该事务的“可见范围”
- 每个数据对象的每个版本需要利用“时间戳”决定版本的“可见范围”

在之前的例子中，每个事务在开始时获取一个时间戳，标识事务的“可见范围”，每个版本在被创建和创建更新版本时，会记录相应时间戳，以确定版本的“可见范围”。

PostgreSQL也采用了类似的思路，在事务开始时，系统会为事务分配一个单调递增的事务号，与前文示例的多版本并发控制协议不同，PostgreSQL并非将事务号作为自己“可见范围”标准，而是利用快照作为标识“可见范围”的数据结构。

PostgreSQL快照数据结构（简化，只列出了最核心的数据项）：

```
snapshot{
  xmin
  xmax
  active_trans_list
}
```

下面解释各数据项的含义：

- xmin表示事务获取该快照时，事务号小于xmin的事务已经全部结束了
- xmax表示事务获取该快照时，事务号大于等于xmax的事务还未开始或者正在运行（还未结束）
- active_trans_list表示事务获取快照时，事务号在 [xmin, xmax) 之间的处在活跃状态的事务列表

快照数据结构记录了事务获取快照时，系统中事务的状态信息，事务可以判断出来在获取快照时哪些事务已经结束（事务号小于xmin，或者事务号不在active_trans_list内）；哪些事务还在运行（事务号大于等于xmax，或者事务号在active_trans_list内）。下面进行实际测试。

输入下述命令：

设置当前会话中事务的隔离级别是serializable。

```
set session characteristics as transaction isolation level serializable;
```

在sql shell中启动一个事务：

```
testdb=# begin;
BEGIN
```

查询当前事务的事务号与快照（不等于885很正常，不同的pg数据库事务号不一定相同）：

```
testdb=# select * from txid_current();
txid_current
-----
      885

testdb=# select * from txid_current_snapshot();
txid_current_snapshot
-----
 885:885:
(1 行记录)
```

表示该事务当前的事务号为885，其获取到的快照是 (xmin:885 xmax:885 active_trans_list: 空)

不要让这个事务结束，我们再启动另外一个sql shell，创建另一个连接postgresql数据库的会话，依次输入如下命令：

```
testdb=# set session characteristics as transaction isolation level serializable;
SET

testdb=# begin;
BEGIN

testdb=# select * from txid_current();
txid_current
-----
      886

testdb=# select * from txid_current_snapshot();
txid_current_snapshot
-----
 885:885:
(1 行记录)

testdb=# end;
COMMIT
```

启动一个事物，该事务的事务号为886（相较上一个递增1），其快照为 (xmin:885 xmax:885 active_trans_list: 空)。
最后输入了"end;"，将该事务结束（提交）。

在第二个sql shell中，继续输入如下命令：

```
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
 txid_current
-----
      887
(1 行记录)

testdb=# select * from txid_current_snapshot();
 txid_current_snapshot
-----
      885:887:885
(1 行记录)
```

上述命令开启新的一个事务，其事务号为887，其快照为 (xmin:885 xmax:887 active_trans_list: 885)。
表示对于该事务来说，事务号小于885的事务已经结束，大于等于887的事务还在运行或者没有开始，
而介于885 - 887之间的事务，886号事务已经结束，885号事务还在运行，因此885在快照的活跃事务列表中。

事务在开始运行时获取快照，在快照中已经结束的事务产生的影响对该事务“可见”，而在快照中处于活跃状态的事务的操作对当前事务“不可见”。

上文中讲解了事务的快照数据结构，接下来讲解postgresql元组上有关事务“可见范围”的数据结构。

postgresql采用多版本并发控制协议，其实现方式与上文提到的基础的多版本并发控制协议类似。每个元组都存在“头部信息”，
里面存放了很多与该元组版本相关的数据结构，此次lab仅关注与并发控制最紧密相关的数据项: t_xmin, t_xmax。
当对元组进行更新时，会创建一个新的元组版本，并更新“头部信息”中与元组可见性相关的数据项，用来进行元组可见性判断。

获取元组的数据可以通过select from where语句查询得到，但是获得元组的“头部信息”需要插件 pageinspect。

首先，将之前开启的事务全部关闭 (end;)，在其中一个sql shell中执行下述命令：

在本次实验的数据库中创建pageinspect扩展（目前阿里云postgresql数据库并不支持创建extension）：

```
testdb=# create extension pageinspect;
CREATE EXTENSION
```

创建用来进行实验的表trans，包含id、data属性，id为主键：

```
testdb=# create table trans (id int primary key, data int);
CREATE TABLE
```

运行下述命令，查找trans表第0块数据页面的内容：

```
testdb=# select * from heap_page_items(get_raw_page('trans',0));
错误:  block number 0 is out of range for relation "trans"
```

报了个错误，不要慌，正常现象，因为trans是刚创建的，还没有插入数据，自然没有数据页面。

接下来开始真正的操作：

首先，开始一个事务，并查看事务号：

```
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
 txid_current
-----
      893
(1 行记录)
```

插入一条数据，(id = 1, data = 1)：

```
testdb=# insert into trans values (1, 1);
INSERT 0 1
```

运行下述命令，查看trans表第0个数据页的内容（没有安装pageinspect插件的没法运行下述命令）：

```
testdb=# select * from heap_page_items(get_raw_page('trans',0));
lp | lp_off | lp_flags | lp_len | t_xmin | t_xmax | t_field3 | t_ctid | t_infomask2 | t_infomask | t_hoff | t_bits | t_oid |
t_data
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
1 | 8160 | 1 | 32 | 893 | 0 | 0 | (0,1) | 2 | 2048 | 24 | | 
\x0100000001000000
(1 行记录)
```

上述输出的内容，t_data存储的是元组的数据，使用16进制表示，可以看到表达的数据内容是(id=1, data=1)。
其余的内容全是元组的“头部信息”，也成为元组的“元信息”，由于内容很多，作用各不相同，此次lab仅关注 "t_xmin""t_xmax"两个与并发控制紧密相关的数据项。
值得注意的是，t_infomask t_infomask2等数据项也参与了事务处理过程，但是原理与实现较为复杂，此次试验不再讲解。感兴趣的同学可以阅读“参考资料”中5、6两本书。

由输出的t_xmin t_max可知，trans表中 id=1 的元组由事务号893的事务创建（t_xmin = 893），且该元组版本是最新的版本，并没有对id=1的元组后续的更新（t_xmax）。

将该事务提交：

```
testdb=# end;
COMMIT
```

重新启动新的事务，新的事务的事务号为894：

```
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
txid_current
-----
894
(1 行记录)
```

对trans数据表id=1的记录进行更新操作，并查看数据页内容：

```
testdb=# update trans set data = 2 where id = 1;
UPDATE 1

testdb=# select t_xmin, t_xmax, t_data from heap_page_items(get_raw_page('trans',0));
t_xmin | t_xmax | t_data
-----+-----+-----
893 | 894 | \x0100000001000000
894 | 0 | \x0100000002000000
(2 行记录)
```

通过输出我们可以看到，postgreSQL更新一个已经存在的元组，会创建一个新的版本。在上述示例中，创建了(id=1, data=2)这个数据版本，且将新版本元数据 t_xmin置为更新事务的事务号894。

同时会将旧的数据版本(id=1, data=1)的元数据“t_xmax”，置为更新事务的事务号894。

下一节中，我们将通过并发的任务之间实际执行的示例，来展示postgreSQL数据库是如何通过事务快照与元组版本的头部信息实现多版本并发控制协议。

在此之前，先清理一下数据：

```
//先将上一个事务提交
testdb=# END;
COMMIT

//删除数据
testdb=# delete from trans;
DELETE 1
```

```
//再看一下trans表0号数据页，发现数据已经标记为删除，但是依然占用着物理空间，我们再继续深度清理一下
testdb=# select t_xmin, t_xmax, t_data from heap_page_items(get_raw_page('trans',0));
 t_xmin | t_xmax | t_data
-----+-----+-----
    893 |    894 | \x0100000001000000
    894 |    895 | \x0100000002000000

//对trans表执行VACUUM，回收物理空间，执行时尽量不要有事务访问trans表所在的数据库
testdb=# vacuum full trans;
VACUUM

//再查看一下，发现数据被彻底清除
testdb=# select t_xmin, t_xmax, t_data from heap_page_items(get_raw_page('trans',0));
错误： block number 0 is out of range for relation "trans"
```

注意：VACUUM进程会物理回收掉数据库中确定不再需要的数据版本（对系统中运行的活跃事务以及未来事务都不可见），在postgresql内部会自动调用VACUUM及时回收物理空间。

由于VACUUM进程运行时会损耗系统性能，手动调用VACUUM FULL会消耗很多资源，本实验由于数据量小且不涉及生产环境，因此不会有影响；但在真实的生产环境中，需要谨慎手动调用VACUUM FULL。

9.4PostgreSQL并发控制

PostgreSQL一共支持三种隔离级别：**读已提交（read committed）**、**可重复读（repeatable read）**、**可串行化（serializable）**。其中，读已提交是PostgreSQL默认使用的隔离级别。

接下来将在不同的隔离级别下，通过并发事务在执行过程中展示事务的快照与数据版本的元数据变化情况，讲解postgresql并发控制的基本原理。

注意：下述实验不考虑事务未成功执行导致回滚的情况，事务均以正确提交（COMMIT）结束事务

PostgreSQL读已提交

首先，关闭之前的终端，重新开启两个sql shell，相当于建立起两个连接postgresql的会话。

在两个终端中均输入下述命令，设置事务的隔离级别为读已提交：

```
testdb=# set session characteristics as transaction isolation level read committed;
SET
```

首先构造测试用的数据，在其中一个终端中输入下述命令，插入(id=1, data=1)：

```
testdb=# begin;
BEGIN
testdb=# insert into trans values (1, 1);
INSERT 0 1

//结束事务（提交）
testdb=# END;
COMMIT
```

根据第二节事务隔离级别中读已提交示例，进行下述操作，展示postgresql在读已提交隔离级别下如何避免脏读，以及为什么无法避免不可重复读：

为了方便表述，用T1 T2分别表示两个sql shell，按照顺序执行下述SQL：

```
T1:
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
 txid_current
-----
    898

T2:
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
 txid_current
-----
    899
```

```

//查看事务899当前的快照
testdb=# select * from txid_current_snapshot();
txid_current_snapshot
-----
898:898:

T1:
testdb=# update trans set data = 2 where id = 1;
UPDATE 1

T2:
testdb=# select * from trans where id = 1;
id | data
---+---
1 | 1

testdb=# select t_xmin, t_xmax, t_data from
heap_page_items(get_raw_page('trans',0));

t_xmin | t_xmax | t_data
-----+-----+-----
897 | 898 | \x0100000001000000
898 | 0 | \x0100000002000000

T1:
testdb=# commit;
COMMIT

T2:
testdb=# commit;
COMMIT

```

在上述SQL序列中，T1先对id=1的记录更新，将data更新为2，然后T2读取id=1的记录。由于T2读取时使用的快照是(xmin:898 xmax:898 active_trans_list: 空)，表示当T2执行读取操作时，事务号小于898的事务对于其已经结束（提交），这些事务对数据库的改变对T2来说是可见的；而对于事务号大于等于898的事务依然活跃（未提交），这些事务对数据库的改变对于事务T2来说是不可见的。

postgresql事务可见性判断核心原则：**事务只读取相对快照来说已经提交的事务对数据库造成的修改。**

因此根据上述可见性原则，事务T2在读取id=1的元组时，它可以“看到”事务号为897的事务对数据库的修改，但是看不到事务号为898事务对数据库的修改。因此(id=1,data=1)的这个元组版本对于事务T2来说并没有被删除，而是“可见的”；与之相反，由于(id=1, data=2)的元组版本的t_xmin=898，表示是被事务号898的事务创建的，因此该元组版本对于事务T2并不可见。综上所述，事务T2的读取id=1的操作依然读取的是旧版本的数据（data=1）。

显然，即使T1最后没有提交，而是回滚（rollback）。由于T2根本看不到T1对于数据库的修改，因此不会发生脏读的数据异常。

但是值得注意的是，postgresql在读已提交隔离级别下，同一个事务内，每次执行一条SQL语句都会重新获取一次快照（当前系统事务的状态）。因此在读已提交隔离级别下，postgresql并不能阻止不可重复读异常。见下述例子：

在两个终端中分别输入下述事务执行序列：

```

T1:
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
txid_current
-----
900

testdb=# select * from txid_current_snapshot();
txid_current_snapshot
-----
900:900:

testdb=# select * from trans where id = 1;
id | data
---+---
1 | 2

T2:

```

```

testdb=# begin;
BEGIN
testdb=# select * from txid_current();
txid_current
-----
          901

testdb=# update trans set data = 3 where id = 1;
UPDATE 1

//T2事务提交
testdb=# commit;
COMMIT

T1:
testdb=# select * from txid_current_snapshot();
txid_current_snapshot
-----
900:902:

testdb=# select * from trans where id = 1;
 id | data
-----+-----
  1 |   3

testdb=# select t_xmin, t_xmax, t_data from heap_page_items(get_raw_page('trans',0));
 t_xmin | t_xmax |      t_data
-----+-----+-----
    897 |    898 | \x0100000001000000
    898 |    901 | \x0100000002000000
    901 |     0 | \x0100000003000000

//T1事务提交
testdb=# end;
COMMIT

```

通过上述示例可以看到，T1首先根据快照(xmin:900 xmax:900 active_trans_list: 空)，判断出(id=1, data=2)版本对当前事务可见（因为(id=1,data=1)版本的t_xmax=898，事务号898的事务在T1当前的快照中已经提交，因此对于T1来说，(id=1,data=1)版本已经“删除”，不可见）。

随后T2开始执行，T2事务号901，执行更新操作后（将id=1的元组data改为3）提交。

后面T1又再次执行读取id=1元组的操作。由于是读已提交隔离级别，事务T1会重新快照，以获得系统最新的事务状态。新快照(xmin:900 xmax:902 active_trans_list: 空)，表示事务号小于900的事务已经结束；事务号大于等于902的事务正在活跃或者还未开始；对于900 901两个中间事务，900是T1自己的事务号，而901由于不在活跃列表中，因此在这个快照中事务号为901的事务已经提交。

因此事务T1第二次执行读取id=1的元组时，使用了与第一次不相同的快照。在新快照中，897 898 901三个事务均已提交，因此(1,1)(1,2)两个版本对于T1均不可见，T2事务并发创建的(1,3)版本对T1可见。所以T2第二次读取了(1,3)，发生了不可重复读异常。

PostgreSQL可重复读

postgresql解决不可重复读异常的思路较为简单，只需要让事务在开始时获取一次快照，并在整个事务运行过程中均使用该快照即可。

可以执行如下事务操作序列：

首先，在两个终端中均执行下述语句，设置隔离级别为可重复读：

```

testdb=# set session characteristics as transaction isolation level repeatable read;
SET

```

在两个终端中分别输入下述事务执行序列：

```

T1:
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
txid_current
-----
          902

testdb=# select * from txid_current_snapshot();

```

```

txid_current_snapshot
-----
902:902:

testdb=# select * from trans where id = 1;
 id | data 
----+-----
  1 |    3 

T2:
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
 txid_current 
-----
          903 

testdb=# update trans set data = 4 where id = 1;
UPDATE 1

//T2事务提交
testdb=# commit;
COMMIT

T1:
testdb=# select * from txid_current_snapshot();
 txid_current_snapshot 
-----
902:902:

testdb=# select * from trans where id = 1;
 id | data 
----+-----
  1 |    3 

testdb=# select t_xmin, t_xmax, t_data from heap_page_items(get_raw_page('trans',0));
 t_xmin | t_xmax | t_data 
-----+-----+-----
    897 |    898 | \x0100000001000000
    898 |    901 | \x0100000002000000
    901 |    903 | \x0100000003000000
    903 |      0 | \x0100000004000000

//T1事务提交
testdb=# end;
COMMIT

```

上述事务操作序列与前面的操作序列相同，唯一区别是事务的隔离级别由读已提交，变为了可重复读；T2的更新操作将data改为4。但是发现，T1前后两次读取id=1的元组data却相同（等于3），并未出现不可重复读异常。

原因在于，T1在最初获取一次快照(xmin:902 xmax:902 active_trans_list: 空)后，整个事务运行期间均使用该快照，因此两次读取均使用相同的快照读取数据。虽然事务T2在T1的两次读取之间，创建了(id=1, data=4)的数据版本，并提交。但是T2第二次读取数据的时候，根据快照(xmin:902 xmax:902 active_trans_list: 空)，事务T2事务号为903，903相对该快照为活跃状态（未提交），因此事务T1读取id=1的元组时，T2对id=1的元组的修改操作对事务T1并不可见。

在可重复读隔离级别下，事务一开始获取一个快照，并在整个事务执行期间均使用同一个快照。这种设计方式也可以避免幻读的数据异常。可见如下示例。

在两个终端中分别输入下述事务执行序列：

```

T1:
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
 txid_current 
-----
          904 

testdb=# select * from txid_current_snapshot();
 txid_current_snapshot 
-----

```

904:904:

```
testdb=# select * from trans where data > 2;
 id | data
```

```
-----+-----
  1 |   4
```

T2:

```
testdb=# begin;
BEGIN
testdb=# select * from txid_current();
 txid_current
```

905

```
testdb=# insert into trans values (2, 5);
INSERT 0 1
```

```
//T2事务提交
testdb=# commit;
COMMIT
```

T1:
testdb=# select * from txid_current_snapshot();
 txid_current_snapshot

904:904:

```
testdb=# select * from trans where data > 2;
 id | data
```

```
-----+-----
  1 |   4
```

```
testdb=# select t_xmin, t_xmax, t_data from heap_page_items(get_raw_page('trans',0));
 t_xmin | t_xmax |      t_data
```

```
-----+-----+-----
  897 |   898 | \x0100000001000000
  898 |   901 | \x0100000002000000
  901 |   903 | \x0100000003000000
  903 |    0 | \x0100000004000000
  905 |    0 | \x0200000005000000
```

```
//T1事务提交
testdb=# end;
COMMIT
```

上述示例中，T1前后进行了两次查询，查询谓词为data > 2。显然，第一次查询符合条件的只有id=1的元组，最新版本data=4对T1可见。后续T2开始新事务，插入了新的元组 (id=2, data=5)，这个新元组在T1查询的谓词条件内 (data > 2)，T2事务提交。随后，T1继续按照data>2谓词条件进行查询。

根据第二节隔离级别定义，在可重复读下无法解决幻象问题，正常来讲T1第二次读data>2的数据，会将(id=2, data=5)也包含在输出结果中。但是pg实际执行结果和第一次执行一样，并不包含新插入的元组。

可以利用事务快照和元组版本的头部信息进行推理一下。在可重复读隔离级别下，T1前后两次查询使用相同的快照，T2插入新的数据，新元组的t_xmin=905。当事务T1进行第二次查询时，由于在快照(xmin:904 xmax:904 active_trans_list: 空)中，事务号905为“活跃事务”，因此事务号905的事务对数据库的修改对于该快照来说不可见（未提交）。因此，元组(2, 5)对事务T1不可见，事务T1第二次查询data>2的元组，结果并不包含(2, 5)。

既然在可重复读隔离级别下，postgresql数据库并不会出现幻读的数据异常，那么为什么并不称其设定为可串行化隔离级别？

原因在于，postgresql在可重复读隔离级别下，会出现写偏序数据异常。

PostgreSQL写偏序异常

起初ANSI SQL92标准定义了三种数据异常，分别为脏读、不可重复读与幻读。但在1995年，有几位数据库领域的专家经过研究发现，在多版本并发控制协议中，会出现“写偏序”异常。

接下来通过示例表明“写偏序”数据异常现象：

在两个终端中依次输入下述事务执行序列（依然是可重复读隔离级别，不用更改）：

```
T1:
testdb=# begin;
BEGIN

testdb=# select count(*) as count from trans where data >= 4;
count
-----
      2

T2:
testdb=# begin;
BEGIN

testdb=# select count(*) as count from trans where data >= 4;
count
-----
      2

//应用层判断，不在sql语句内，不要在sql shell中执行
if count == 2

//判断成立，将下述语句输入到sql shell中
T1:
testdb=# update trans set data = 3 where id = 1;
UPDATE 1

testdb=# end;
COMMIT

if count == 2

T2:
testdb=# update trans set data = 3 where id = 2;
UPDATE 1
testdb=# end;
COMMIT
```

上述示例中，事务T1执行的逻辑是：如果trans表中data>=4的元组数量等于2，那么T1就将id=1的元组data更新为3；

事务T2执行的逻辑是：如果trans表中data>=4的元组数量等于2，那么T2就将id=2的元组更新为3。

if判断并不在sql语句中而在应用逻辑层实现。

由于trans表初始数据状态是：(1,4) (2,5)，那么两个事务开始运行查询时，得到的结果均为2，由于符合条件，后续两个事务分别会继续执行更新操作。

上述事务运行结束后，trans表的数据状态如下：

```
testdb=# select * from trans;
 id | data 
----+-----
  1 |    3 
  2 |    3
```

会发现，经过T1 T2并发执行后，数据从(1,4) (2,5) 转化为了(1,3) (2,3)。

根据可串行化调度的定义：**事务并发执行的结果与事务按照某种串行顺序执行后的数据结果相同。**

显然按照这个定义，应存在一个串行执行顺序 T1->T2 或者 T2->T1，其中一个运行结果等于(1,3) (2,3)。

但是，我们的大脑中串行执行一下两个事务：

初始数据为(1,4) (2,5)，如果先运行T1，那么结果是data>=4的元组数量等于2，将(1,4)更新为(1,3)，T1提交；后续继续执行T2，data>=4的元组数量等于1，并不执行更新，T1结束。最终的数据状态为(1,3) (2,5)。

反过来，如果先执行T2，后执行T1。那么最终的数据状态是(1,4) (2,3)。

由此可见，上述示例中T1与T2并发执行的结果并不等价于任何一种串行执行的结果，因此不满足可串行化调度。

而上述出现的数据异常现象，称为**写偏序异常**。

PostgreSQL可串行化

上文介绍了PostgreSQL在可重复读隔离级别下，虽然避免了脏读、不可重复读与幻读，但是存在写偏序异常。

下面通过示例展示PostgreSQL在可串行化隔离级别下，是如何处理写偏序异常的。

首先利用下述更新，将数据会恢复到初始状态(1,4) (2,5)：

```
testdb=# update trans set data = 4 where id = 1;
UPDATE 1
testdb=# update trans set data = 5 where id = 2;
UPDATE 1
```

接下来，在两个终端中均输入下述命令，设置事务隔离级别为可串行化：

```
testdb=# set session characteristics as transaction isolation level serializable;
SET
```

在两个终端中依次输入下述事务执行序列：

```
T1:
testdb=# begin;
BEGIN

testdb=# select count(*) as count from trans where data >= 4;
count
-----
      2

//应用层判断，不在sql语句内，不要在sql shell中执行
if count == 2

//判断成立，将下述语句输入到sql shell中
T1:
testdb=# update trans set data = 3 where id = 1;
UPDATE 1

//事务正常提交
testdb=# end;
COMMIT

T2:
testdb=# begin;
BEGIN

testdb=# select count(*) as count from trans where data >= 4;
count
-----
      2

if count == 2

T2:
testdb=# update trans set data = 3 where id = 2;
错误： 由于多个事务间的读/写依赖而无法串行访问
描述： Reason code: Canceled on identification as a pivot, during write.
提示： 该事务如果重试，有可能成功。

//事务选择回滚
testdb=!# end;
ROLLBACK
```

可以看到，在可串行化隔离级别下，PostgreSQL会选择将其中一个事务回滚（ROLLBACK），阻止写偏序异常的发生。

虽然多版本并发控制机制存在写偏序数据异常，但并非所有采用了多版本并发控制机制的商业数据库都会阻止写偏序异常的发生，例如，大名鼎鼎的ORACLE就没有提供避免写偏序异常的能力。

练习题

题目1 总结PostgreSQL在不同隔离级别下，会出现哪些数据异常现象，能够禁止哪些数据异常

mx:

隔离级别	脏读	不可重复读	幻读	写偏序
读已提交	能禁止	可能出现	可能出现	可能出现
可重复读	能禁止	能禁止	可能出现	可能出现
可串行化	能禁止	能禁止	能禁止	能禁止

题目2 请总结多版本并发控制协议与两阶段封锁协议相比，主要的优势在哪里

mx: 总体来说，两阶段封锁协议效率低下，尤其是对于只读数据。对数据的上锁会大大降低数据库读写的效率。而多版本并发控制协议能在保证一定正确性的前提下，提供多个任务并发的协议，大大提高了数据读写的效率。上锁是一种绝对安全的做法，但在效率面前，适当放弃一些正确性要求往往是必要的。而且对于可串行化的隔离级别，大部分的数据异常问题都可以得到解决，只不过协议内容更加复杂。因此，在能够保证事务正确性的前提下，多版本并发控制协议效率要比两阶段封锁协议高很多。

ORM

ORM: Object Relation Mapping，最初主要描述的是程序中的Object对象和关系型数据库中Relation关系(表)之间的映射关系，目前来说也是描述程序中对象和数据库中数据记录之间的映射关系的统称，是一种进行程序和数据库之间数据持久化的一种编程思想。特点是操纵Python对象而不是SQL查询，也就是在代码层面考虑的是对象，而不是SQL，体现的是一种程序化思维，这样使得Python程序更加简洁易读。

增删改操作

常规情况下，软件程序中的ORM操作主要有四个操作场景：增、删、改、查。核心操作一般会区分为：增删改、查询

增加操作: 程序中存在的一个对象Object数据，通过[ORM]核心模块进行增加的函数定义将对象保存到数据库的操作过程；

如：注册操作中，通过用户输入的账号密码等信息创建了一个独立的对象，通过 `add()` 函数将对象增加保存到数据库中，数据库中就存在用户这个对象数据了。

修改操作: 程序中存在的一个对象Object数据，有自己的id编号(可以是程序中自行赋值定义、更多的操作是从数据库中查询出来存在的一个对象)，通过[ORM]核心模块进行修改函数的定义将对象改变的数据更新到数据库中已经存在的记录中的过程；

如:用户更改登录密码操作时，根据程序中查询得到的一个用户[id编号、账号、密码、..]，在程序中通过改变其密码属性数据，然后通过 `update()` 函数将改变的数更新保存到数据库中，数据库中原来的数据就发生了新的改变。

删除操作: 程序中存在的一个对象或者已知的id编号，通过主键编号或者对象的任意属性进行数据库中数据记录的删除的操作过程；

如：管理员删除某个会员账号的操作，通过获取要删除会员的账号，然后通过 `delete()` 函数将要删除的会员信息告知数据库执行删除操作，数据库中的某条存在的数据记录就被删除掉了。

sqlalchemy

SQLAlchemy 是一个Python 的SQL 工具包以及数据库对象映射框架。它包含整套企业级持久化模式，专门为高效和高性能的数据库访问。

```
pip install SQLAlchemy
pip install psycopg2
import sqlalchemy
from sqlalchemy import Column, String, create_engine, Integer, Text, Date
from sqlalchemy.orm import sessionmaker
from sqlalchemy.ext.declarative import declarative_base
import time

from sqlalchemy import create_engine
engine = create_engine("postgresql://stu10205501417:stu10205501417@dase-cdms-2022-pub.pg.rds.aliyuncs.com:5432/stu10205501417",
    echo=True,
    pool_size=8,
    pool_recycle=60*30
)
```

建立连接

```

from sqlalchemy import create_engine
engine = create_engine("postgresql://stu10205501417:Stu10205501417@dase-cdms-2022-pub.pg.rds.aliyuncs.com:5432/stu10205501417",
    echo=True,
    pool_size=8,
    pool_recycle=60*30
)

```

session 用于创建程序与数据库之间的对话.

```

from sqlalchemy.orm import sessionmaker
# 创建session
DbSession = sessionmaker(bind=engine)
session = DbSession()

```

session 的常见用法:

1. commit: 提交了一个事务
2. rollback: 回滚
3. close: 关闭

创建表格

declarative_base()是sqlalchemy内部封装的一个方法, 通过其构造一个基类, 这个基类和它的子类, 可以将Python类和数据库表关联映射起来。数据库表模型类通过tablename和表关联起来, Column表示数据表的列。

```

from sqlalchemy.ext.declarative import declarative_base
# 创建对象的基类:
Base = declarative_base()

```

```

# 定义User对象:
class User(Base):
    # 表的名字:
    __tablename__ = 'users'

    # 表的结构:
    id = Column(Integer, autoincrement=True, primary_key=True, unique=True, nullable=False)
    name = Column(String(50), nullable=False)
    sex = Column(String(4), nullable=False)
    nation = Column(String(20), nullable=False)
    birth = Column(String(8), nullable=False)
    id_address = Column(Text, nullable=False)
    id_number = Column(String(18), nullable=False)
    creator = Column(String(32))
    create_time = Column(String(20), nullable=False)
    updater = Column(String(32))
    update_time = Column(String(20), nullable=False,
        default=time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()),
        onupdate=time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()))
    comment = Column(String(200))

def createTable():
    # 创建所有继承于Base的类对应的表
    Base.metadata.create_all(engine)

createTable()

```

插入数据

```

# 插入操作
def insertData():

    # 创建会话
    session = DbSession()

    # 创建新用户对象:
    local_time = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime())
    new_user = User(name='mdotdot', sex='女', nation='汉',
        birth='19981021', id_address='ECNU', id_number='441242142142',
        create_time=local_time)

```

```
new_user1 = User(name='xdot', sex='男', nation='汉',
birth='19990110', id_address='ECNU', id_number='451242142142',
create_time=local_time)

# 添加到session:
session.add(new_user)
session.add(new_user1)
# 提交即保存到数据库:
session.commit()

# 关闭session:
session.close()

insertData()
```

查询数据

SQL 与 SQLalchemy 的写法区别为:

- query : 对应 SELECT xxx FROM xxx
- filter/filter_by : 对应 WHERE , fillter 可以进行比较运算(==, >, < ...)来对条件进行灵活的运用, 不同的条件用逗号分割, filter_by 只能指定参数传参来获取查询结果。
- limit : 对应 limit()
- order by : 对应 order_by()
- group by : 对应 group_by()

返回结果数量可以有以下两种方式:

- all()
- 查询所有
 - 返回一个列表对象

- first()
- 查询第一个符合条件的对象
 - 返回一个对象

在ORM中, 查询也有和SQL类似的关键字

like	session.query(Person).filter(Person.desc.like("活%")).all()
not like	session.query(Person).filter(Person.desc.notlike("活%")).all()
is(等价于==)	session.query(Person).filter(Person.username.is_(None)).all(), session.query(Person).filter(Person.username == None).all()
isnot(等价于!=)	session.query(Person).filter(Person.username.isnot(None)).all(), session.query(Person).filter(Person.username != None).all()
正则查询	session.query(Person).filter(Person.password.op("regexp")(r"^\u4e00-\u9fa5+")).all()
count	session.query(Person).filter(Person.desc.like("活%")).count()
in	session.query(Person).filter(Person.username.in_(['Mark', 'Tony'])).all()
not in(等价于~in)	session.query(Person).filter(Person.username.notin_(['Mark', 'Tony'])).all(), session.query(Person).filter(~Person.username.in_(['Mark', 'Tony'])).all()
AND(导入and_)	more_person = session.query(Person).filter(and_(Person.password=='123456',Person.desc=="可爱")).all()
OR(导入or_)	session.query(Person).filter(or_(Person.password=='123456',Person.desc=="活泼")).all()
limit	session.query(Person).filter(Person.desc.notlike("活%")).limit(1).all()
offset	session.query(Person).filter(Person.desc.like("活%")).offset(1).all()
order_by (asc正序)	session.query(Person).order_by(Person.username.desc()).all()
group_by	session.query(Person).group_by(Person.desc).all()
between	session.query(Protocols.protocolName).filter(Protocols.id.between(1, 3)).all()

聚合函数

关键字	示例
count	session.query(Person.password, func.count(Person.id)).group_by(Person.password).all()
sum	session.query(Person.password, func.sum(Person.id)).group_by(Person.password).all()
max	session.query(Person.password, func.max(Person.id)).group_by(Person.password).all()
min	session.query(Person.password, func.min(Person.id)).group_by(Person.password).all()
having	session.query(Person.password, func.count(Person.id)).group_by(Person.password).having(func.count(Person.id) > 1).all()

```
#查询所有数据
def find_all():
    # 创建Session
    session = DbSession()

    user = session.query(User).all()
    for i in user:
        print('id:', i.id)
        print('name:', i.name)
        print('id_address:', i.id_address)
        print('id_number:', i.id_number)

    session.close() # 关闭Session

find_all()
```

```
# 查询操作
def selectData():
    # 创建Session
    session = DbSession()

    # 创建Query查询, filter是where条件, 最后调用first()返回唯一行, 如果调用all()则返回所有行:
    user = session.query(User).filter(User.id == '1' and User.name == 'mdotdot').first()

    if user:
        print('name:', user.name)
        print('id_address:', user.id_address)

    session.close() # 关闭Session

selectData()
```

- 还可以将查询的参数单独写:

```
# 创建Session
session = DbSession()
filter = (User.name=='mdotdot')
user = session.query(User).filter(filter).first()
print(user.name)
session.close()
```

- 适用于批量修改

```
session.query(User).filter_by(name = "mdotdot").update({'name': "Jack"})
session.commit() # 提交即保存到数据库
```

- 适用于获取对象的值,进行操作之后修改

```
# 更新操作
def updateData():
    session = DbSession() # 创建会话

    users = session.query(User).filter(User.name=="Jack").first()# 查询条件

    if users:
        users.id_number = "abcd" # 更新操作
        session.add(users) # 添加到会话
```

```
session.commit() # 提交即保存到数据库

session.close() # 关闭会话

updateData()
```

```
find_all()
```

find_all()查询一下，看看更改是否发生

- 直接将删除语句写成一

```
delete_query = session.query(User).filter(User.name=='xdot').delete()
session.commit() # 提交会话
```

- 查找到数据后再删除

```
# 删除操作
def deleteData():
    session = DbSession() # 创建会话

    delete_users = session.query(User).filter(User.id == "1").first()
    if delete_users:
        session.delete(delete_users)
        session.commit()

    session.close() # 关闭会话

deleteData()
```

删除表格

```
def dropTable():
    sql = 'DROP TABLE IF EXISTS users;'
    result = engine.execute(sql)

dropTable()
```

删除所有表

```
Base.metadata.drop_all(engine)
```

建立会话

```
from sqlalchemy.orm import sessionmaker
# 创建session
DbSession = sessionmaker(bind=engine)
session = DbSession()
```

表格创建

```
create table student(
s_id varchar(10),
s_name varchar(20),
s_age date,
s_sex varchar(10)
);

create table course(
c_id varchar(10),
c_name varchar(20),
t_id varchar(10)
);

create table teacher (
t_id varchar(10),
t_name varchar(20)
);
```

```
create table score (
s_id varchar(10),
c_id varchar(10),
score integer );
```

```
from sqlalchemy.ext.declarative import declarative_base
# 创建对象的基类：
Base = declarative_base()

# 定义Student对象：
class Student(Base):
    # 表的名字：
    __tablename__ = 'student'

    # 表的结构：
    s_id = Column(String(10), primary_key=True, unique=True, nullable=False)
    s_name = Column(String(20), nullable=False)
    s_age = Column(Date, nullable=False)
    s_sex = Column(String(10), nullable=False)

# 定义Course对象：
class Course(Base):
    # 表的名字：
    __tablename__ = 'course'

    # 表的结构：
    c_id = Column(String(10), primary_key=True, unique=True, nullable=False)
    c_name = Column(String(20), nullable=False)
    t_id = Column(String(10), nullable=False)
    .....

def createTable():
    # 创建所有继承于Base的类对应的表
    Base.metadata.create_all(engine)

createTable()
```

插入数据

思考：怎样写代码可以批量插入数据

```
student_data = [
    ('01', '赵雷', '1990-01-01', '男'),
    ('02', '钱电', '1990-12-21', '男'),
    ('03', '孙凤', '1990-05-20', '男')
]
course_data = [
    ('01', '语文', '02'),
    ('02', '数学', '01'),
    ('03', '英语', '03'),
]
teacher_data = [
    ('01', '张三'),
    ('02', '李四'),
    ('03', '王五'),
]

# 为每个表构造一个批量对象生成并插入数据函数
def student_insert_many(data):
    session = DbSession()

    for s in data:
        stu = Student(s_id=s[0], s_name=s[1], s_age=s[2], s_sex=s[3])
        session.add(stu)

    session.commit()
    session.close()

def course_insert_many(data):
    session = DbSession()
```



```

for c in data:
    course = Course(c_id=c[0], c_name=c[1], t_id=c[2])
    session.add(course)

session.commit()
session.close()

def teacher_insert_many(data):
    session = DbSession()

    for t in data:
        teacher = Teacher(t_id=t[0], t_name=t[1])
        session.add(teacher)

    session.commit()
    session.close()

def score_insert_many(data):
    session = DbSession()

    for s in data:
        score = Score(s_id=s[0], c_id=s[1], score=s[2])
        session.add(score)

    session.commit()
    session.close()

```

```

student_insert_many(student_data)
course_insert_many(course_data)
teacher_insert_many(teacher_data)
score_insert_many(score_data)

```

1.查询学生中的所有女生，并将名字按降序排序

```

def task1():
    stu = session.query(Student).filter(Student.s_sex == '女').order_by(Student.s_name.desc()).all()
    for g in stu:
        print('name:', g.s_name)
task1()

```

2.查询"01"课程中成绩最高的5位同学的id和成绩

```

def task2():
    result = session.query(Score).filter(Score.c_id == '01').order_by(Score.score.desc()).limit(5).all()
    for r in result:
        print('s_id:', r.s_id)
        print('score', r.score)
task2()

```

3.查询出生年份在1990年的同学(注意: s_age的类型为date)

```

from sqlalchemy import func, extract, and_, or_

def task3():
    result = session.query(Student).filter(extract('Y', Student.s_age) == 1990).all()
    for r in result:
        print('\ns_id', r.s_id)
        print('s_name', r.s_name)
        print('s_age', r.s_age)
        print('s_sex', r.s_sex)
task3()

```

4.查询每位同学一共选择了几门课和总成绩

```
def task4():
    result = session.query(Score.s_id, func.count(Score.c_id), func.sum(Score.score)).group_by(Score.s_id).all()
    for r in result:
        print('\ns_id:', r.s_id)
        print('course count:', r[1])
        print('score sum:', r[2])

task4()
```

5. 查询01课程或02课程成绩大于85的同学id

```
def task5():
    # 投影自带去重
    result = session.query(Score.s_id).filter(and_(or_(Score.c_id=='01', Score.c_id=='02'), Score.score>85)).all()
    for r in result:
        print('\ns_id:', r.s_id)

task5()
```

多表连接

```
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy import Enum, ForeignKey, UniqueConstraint
from sqlalchemy.orm import relationship
Base = declarative_base()

class StudentsNumberInfo(Base):
    """学号表"""
    __tablename__ = "studentsNumberInfo"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    number = Column(Integer, nullable=False, unique=True, comment="学生编号")
    admission = Column(Date, nullable=False, comment="入学时间")
    graduation = Column(Date, nullable=False, comment="毕业时间")

class TeachersInfo(Base):
    """教师表"""
    __tablename__ = "teachersInfo"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    number = Column(Integer, nullable=False, unique=True, comment="教师编号")
    name = Column(String(64), nullable=False, comment="教师姓名")
    gender = Column(String(1), nullable=False, comment="教师性别")
    age = Column(Integer, nullable=False, comment="教师年龄")

class ClassesInfo(Base):
    """班级表"""
    __tablename__ = "classesInfo"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    number = Column(Integer, nullable=False, unique=True, comment="班级编号")
    name = Column(String(64), nullable=False, unique=True, comment="班级名称")
    # 一对一关系必须为连接表的连接字段创建UNIQUE的约束，这样才能是一对一，否则是一对多
    fk_teacher_id = Column(
        Integer,
        ForeignKey(
            "teachersInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE",
        ),
        nullable=False,
        unique=True,
        comment="班级负责人"
    )

# 下面这2个均属于逻辑字段，适用于正反向查询。在使用ORM的时候，我们不必每次都进行JOIN查询，而恰好正反向的查询使用频率会更高
# 这种逻辑字段不会在物理层面上创建，它只适用于查询，本身不占据任何数据库的空间
# sqlalchemy的正反向概念与Django有所不同，Django是外键字段在那边，那边就作为正
# 而sqlalchemy是relationship字段在那边，那边就作为正
# 比如班级表拥有 relationship 字段，而老师表不曾拥有
# 那么用班级表的这个relationship字段查老师时，就称为正向查询
# 反之，如果用老师来查班级，就称为反向查询
# 另外对于这个逻辑字段而言，根据不同的表关系，创建的位置也不一样：
#   - 1 TO 1: 建立在任意一方均可，查询频率高的一方最好
#   - 1 TO M: 建立在M的一方
```

```

# - M TO M: 中间表中建立2个逻辑字段，这样任意一方都可以先反向，再正向拿到另一方
# - 遵循一个原则，ForeignKey建立在那个表上，那个表上就建立relationship
# - 有几个ForeignKey，就建立几个relationship
# 总而言之，使用ORM与原生SQL最直观的区别就是正反向查询能带来更高的代码编写效率，也更加简单
# 甚至我们可以不用外键约束，只创建这种逻辑字段，让表与表之间的耦合度更低，但是这样要避免脏数据的产生
# 班级负责人，这里是一对一关系，一个班级只有一个负责人
leader_teacher = relationship(
    # 正向查询时所链接的表，当使用 classesInfo.leader_teacher 时，它将自动指向fk的那一条记录
    "TeachersInfo",
    # 反向查询时所链接的表，当使用 teachersInfo.leader_class 时，它将自动指向该老师所管理的班级
    backref="leader_class",
)
)
class ClassesAndTeachersRelationship(Base):
    """任教老师与班级的关系表"""
    __tablename__ = "classesAndTeachersRelationship"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    # 中间表中注意不要设置单列的UNIQUE约束，否则就会变为一对一
    fk_teacher_id = Column(
        Integer,
        ForeignKey(
            "teachersInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE",
        ),
        nullable=False,
        comment="教师记录"
    )
    fk_class_id = Column(
        Integer,
        ForeignKey(
            "classesInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE",
        ),
        nullable=False,
        comment="班级记录"
    )
    )
# 多对多关系的中间表必须使用联合唯一约束，防止出现重复数据
__table_args__ = (
    UniqueConstraint("fk_teacher_id", "fk_class_id"),
)
# 逻辑字段
# 给班级用的，查看所有任教老师
mid_to_teacher = relationship(
    "TeachersInfo",
    backref="mid",
)
# 给老师用的，查看所有任教班级
mid_to_class = relationship(
    "ClassesInfo",
    backref="mid"
)
)
class StudentsInfo(Base):
    """学生信息表"""
    __tablename__ = "studentsInfo"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    name = Column(String(64), nullable=False, comment="学生姓名")
    gender = Column(String(1), nullable=False, comment="学生性别")
    age = Column(Integer, nullable=False, comment="学生年龄")
    # 外键约束
    # 一对一关系必须为连接表的连接字段创建UNIQUE的约束，这样才能是一对一，否则是一对多
    fk_student_id = Column(
        Integer,
        ForeignKey(
            "studentsNumberInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE"
        ),
        nullable=False,
        comment="学生编号"
    )
    )
# 相比于一对一，连接表的连接字段不用UNIQUE约束即为多对一关系

```

```

fk_class_id = Column(
    Integer,
    ForeignKey(
        "classesInfo.id",
        ondelete="CASCADE",
        onupdate="CASCADE"
    ),
    comment="班级编号"
)
# 逻辑字段
# 所在班级，这里是一对多关系，一个班级中可以有多名学生
from_class = relationship(
    "ClassesInfo",
    backref="have_student",
)
# 学生学号，这里是一对一关系，一个学生只能拥有一个学号
number_info = relationship(
    "StudentsNumberInfo",
    backref="student_info",
)
)
if __name__ == "__main__":
    # 删除表
    Base.metadata.drop_all(engine)
    # 创建表
    Base.metadata.create_all(engine)

```

插入数据，略

```

Session = sessionmaker(bind=engine)
session = scoped_session(Session)
result = session.query(
    StudentsInfo.name,
    StudentsNumberInfo.number,
    ClassesInfo.number
).join(
    StudentsNumberInfo,
    StudentsInfo.fk_student_id == StudentsNumberInfo.id
).join(
    ClassesInfo,
    StudentsInfo.fk_class_id == ClassesInfo.id
).all()
print(result)
# [('Jack', 160201, 1602), ('Tom', 160101, 1601), ('Mary', 160301, 1603), ('Anna', 160102, 1601), ('Bobby', 160202, 1602)]
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()

```

LEFT JOIN

left join只需要在每个JOIN中指定isouter关键字参数为True即可：

```

session.query(
    左表.字段,
    右表.字段
)
.join(
    右表,
    链接条件,
    isouter=True
).all()

```

RIGHT JOIN

需要换表的位置，SQLAlchemy本身并未提供RIGHT JOIN，所以使用时一定要注意驱动顺序，小表驱动大表：

```
session.query(
    左表.字段,
    右表.字段
)
.join(
    左表,
    链接条件,
    isouter=True
).all()
```

子查询

子查询使用subquery()实现，如下所示，查询每个班级中年龄最小的人：

```
# 获取链接池、ORM表对象
from sqlalchemy import func
# 子查询中所有字段的访问都需要加上c的前缀
# 如 sub_query.c.id、 sub_query.c.name等
sub_query = session.query(
    # 使用label()来为字段AS一个别名
    # 后续访问需要通过sub_query.c.alias进行访问
    func.min(StudentsInfo.age).label("min_age"),
    ClassesInfo.id,
    ClassesInfo.name
).join(
    ClassesInfo,
    StudentsInfo.fk_class_id == ClassesInfo.id
).group_by(
    ClassesInfo.id
).subquery()

result = session.query(
    StudentsInfo.name,
    sub_query.c.min_age,
    sub_query.c.name
).join(
    sub_query,
    sub_query.c.id == StudentsInfo.fk_class_id
).filter(
    sub_query.c.min_age == StudentsInfo.age
)

print(result.all())
# [('Jack', 17, 'one year two class'), ('Mary', 16, 'one year three class'), ('Anna', 17, 'one year one class')]

# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

正反查询

正反查询

上面我们都是通过JOIN进行查询的，实际上我们也可以通过逻辑字段relationship进行查询。

正向查询

下面是正向查询的示例，正向查询是指从有relationship逻辑字段的表开始查询：

```
# 查询所有学生的所在班级，我们可以通过学生的from_class字段拿到其所在班级
# 另外，对于学生来说，班级只能有一个，所以have_student应当是一个对象
# 获取链接池、ORM表对象
students_lst = session.query(
    StudentsInfo
).all()
for row in students_lst:
    print(f"""
        student name : {row.name}
        from : {row.from_class.name}
    """)

# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

反向查询

下面是反向查询的示例，反向查询是指从没有relationship逻辑字段的表开始查询：

```
# 查询所有班级中的所有学生，学生表中有relationship，并且它的backref为have_student，所以我们可以通过班级.have_student来获取所有学生记录
# 另外，对于班级来说，学生可以有多个，所以have_student应当是一个序列
classes_lst = session.query(
    ClassesInfo
).all()
for row in classes_lst:
    print("class name :", row.name)
    for student in row.have_student:
        print("student name :", student.name)
session.close()
```

总结，正向查询的逻辑字段总是得到一个对象，反向查询的逻辑字段总是得到一个列表。

反向方法

使用逻辑字段relationship可以直接对一些跨表记录进行增删改查。

由于逻辑字段是一个类似于列表的存在（仅限于反向查询，正向查询总是得到一个对象），所以列表的绝大多数方法都能用。

```
<class 'sqlalchemy.orm.collections.InstrumentedList'>
- append()- clear()- copy()- count()- extend()- index()- insert()- pop()- remove()- reverse()- sort()
```

下面不再进行实机演示，因为我们上面的几张表中做了很多约束。

一下代码只是举例，运行时不会成功

```
# 比如
# 给老师增加班级
result = session.query(Teachers).first()
# extend方法:
result.re_class.extend([
    Classes(name="三年级一班",),
    Classes(name="三年级二班",),
])
# 比如
# 减少老师所在的班级
result = session.query(Teachers).first()
# 待删除的班级对象,集合查找比较快
delete_class_set = {
    session.query(Classes).filter_by(id=7).first(),
    session.query(Classes).filter_by(id=8).first(),
}
# 循环老师所在的班级
# remove方法:
for class_obj in result.re_class:
    if class_obj in delete_class_set:
        result.re_class.remove(class_obj)

# 比如
# 清空老师所任教的所有班级
# 拿出一个老师
result = session.query(Teachers).first()
```

```
result.re_class.clear()
```

```
NameError                                Traceback (most recent call last)
Input In [16], in ()
      1 # 一下代码只是举例，运行时不会成功
      2
      3
      4 # 比如
      5 # 给老师增加班级
----> 6 result = session.query(Teachers).first()
      7 # extend方法:
      8 result.re_class.extend([
      9     classes(name="三年级一班",),
     10     classes(name="三年级二班",),
     11 ])

NameError: name 'Teachers' is not defined
```

综合练习

(1) 查看每个班级共有多少学生:

```
#JOIN查询:

# 获取链接池、ORM表对象
from sqlalchemy import func
result = session.query(
    ClassesInfo.name,
    func.count(StudentsInfo.id)
).join(
    StudentsInfo,
    ClassesInfo.id == StudentsInfo.fk_class_id
).group_by(
    ClassesInfo.id
).all()
print(result)
# [('one year one class', 2), ('one year two class', 2), ('one year three class', 1)]
session.remove(), 它将回收该链接
session.close()
```

```
#正反查询:
result = {}
class_lst = session.query(
    ClassesInfo
).all()
for row in class_lst:
    for student in row.have_student:
        count = result.setdefault(row.name, 0)
        result[row.name] = count + 1
print(result.items())
# dict_items([('one year one class', 2), ('one year two class', 2), ('one year three class', 1)])
session.remove(), 它将回收该链接
session.close()
```

(2) 查看每个学生的入学、毕业年份以及所在的班级名称:

```
#JOIN查询:
result = session.query(
    StudentsNumberInfo.number,
    StudentsInfo.name,
    ClassesInfo.name,
    StudentsNumberInfo.admission,
    StudentsNumberInfo.graduation
).join(
    StudentsInfo,
    StudentsInfo.fk_class_id == ClassesInfo.id
).join(
    StudentsNumberInfo,
    StudentsNumberInfo.id == StudentsInfo.fk_student_id
```

```

).order_by(
    StudentsNumberInfo.number.asc()
).all()
print(result)
session.remove(), 它将回收该链接
session.close()

```

```

#正反查询:
result = []
student_lst = session.query(
    StudentsInfo
).all()
for row in student_lst:
    result.append((
        row.number_info.number,
        row.name,
        row.from_class.name,
        row.number_info.admission,
        row.number_info.graduation
    ))
print(result)
session.remove(), 它将回收该链接
session.close()

```

3) 查看David所教授的学生中年龄最小的学生:

```

#JOIN查询
result = session.query(
    TeachersInfo.name,
    StudentsInfo.name,
    StudentsInfo.age,
    ClassesInfo.name
).join(
    ClassesInfo,
    StudentsInfo.fk_class_id == ClassesInfo.id
).join(
    ClassesAndTeachersRelationship,
    ClassesInfo.id == ClassesAndTeachersRelationship.fk_class_id
).join(
    TeachersInfo,
    ClassesAndTeachersRelationship.fk_teacher_id == TeachersInfo.id
).filter(
    TeachersInfo.name == 'David'
).order_by(
    StudentsInfo.age.asc()
).first()
print(result)

session.close()

```

```

#正反查询:
result = []
student_lst = session.query(
    StudentsInfo
).all()
teacher_lst = session.query(
    TeachersInfo
).all()
teacher_class_lst = session.query(
    ClassesAndTeachersRelationship
).all()

class_id = []
for row in teacher_lst:
    if row.name == 'David':
        class_id = [x.fk_class_id for x in row.mid]
for row in student_lst:
    if row.from_class.id in class_id:
        result.append((
            'David',

```



```

        row.name,
        row.age,
        row.from_class.name
    ))

result.sort(key = lambda x: x[2])
print(result[0])

session.close()

```

4) 查看每个班级的负责人是谁, 以及任课老师都有谁:

```

# JOIN
from sqlalchemy import func

sub_query = session.query(
    ClassesInfo.id,
    ClassesInfo.name,
    func.string_agg(TeachersInfo.name, ',').label("teaching_teachers")
).join(
    ClassesAndTeachersRelationship,
    ClassesInfo.id == ClassesAndTeachersRelationship.fk_class_id
).join(
    TeachersInfo,
    ClassesAndTeachersRelationship.fk_teacher_id == TeachersInfo.id
).group_by(
    ClassesInfo.id
).subquery()

result = session.query(
    ClassesInfo.name,
    TeachersInfo.name,
    sub_query.c.teaching_teachers
).join(
    sub_query,
    sub_query.c.id == ClassesInfo.id
).join(
    TeachersInfo,
    ClassesInfo.fk_teacher_id == TeachersInfo.id
).all()
print(result)

session.close()

```

```

# 正反查询
result = []
class_lst = session.query(
    ClassesInfo
).all()
teacher_lst = session.query(
    TeachersInfo
).all()

for c in class_lst:
    c_teacher_name = None
    t_teachers_id = [x.fk_teacher_id for x in c.mid]
    t_teachers = []
    for t in teacher_lst:
        if c.fk_teacher_id == t.id:
            c_teacher_name = t.name
        if t.id in t_teachers_id:
            t_teachers.append(t.name)
    result.append((c.name, c_teacher_name, ','.join(t_teachers)))
print(result)

session.close()

```

原生SQL

查看执行命令

如果一条查询语句是filter()结尾，则该对象的**str**方法会返回格式化后的查询语句：

```
print(session.query(StudentsInfo).filter())
```

```
SELECT "studentsInfo".id AS "studentsInfo_id", "studentsInfo".name AS "studentsInfo_name"  
FROM "studentsInfo"
```

这只是一个示例